

基于沙漠游戏的优化建模

2026 年 4 月 7 日

马元昊 倪皓玥 曹宇丰

摘要

“穿越沙漠”是一款生存型游戏，需要选择最佳游戏策略，使玩家在保证到达终点的情况下尽可能获得更多资金。针对“穿越沙漠”问题中不同信息条件下的路径规划、资源配置与多人协同行动决策，本文以剩余资金最大化为核心目标，建立了单人确定性决策模型、单人随机决策模型以及多人协同优化模型，对不同关卡情形下的最优策略进行了系统研究。首先，根据地图中各区域的邻接关系构建无向图模型，利用 Dijkstra 算法求解起点、村庄、矿山与终点之间的最短路径，为后续策略优化提供路径基础。

对于问题一，在已知全时段天气信息的条件下，综合考虑负重约束、资源消耗约束、资金约束及到达终点的时限约束，建立以最终剩余资金最大化为目标的动态规划模型。模型对“直接前往终点”与“经矿山挖矿增益后再前往终点”两类策略进行统一刻画，使用动态规划，通过状态转移递推求解各阶段的最优行动方案，从而得到单人情形下的全局最优决策。

对于问题二，在仅知道当天天气、未来天气具有随机性的条件下，将天气演化过程引入马尔可夫决策思想，建立基于状态转移概率的动态决策模型。模型结合当前位置、剩余资源、剩余天数及资金状态，对“前进、停留、挖矿、补给”等动作进行滚动优化，从而获得不确定天气条件下的最优策略，提高了模型在动态环境中的适应能力。

对于问题三，考虑多人同时参与时因同行、同购和同矿带来的资源消耗放大与收益分摊效应，我们选择合作而非竞争的博弈模式，在单人模型基础上建立多人协同优化框架。进一步采用蒙特卡洛方法模拟天气过程，对不同随机天气样本下的多人协同行为进行数值实验，从而评估策略的稳定性与收益表现。结果表明，所建模型能够较好统筹路径选择、资源补给与行动安排，在确定与不确定天气条件下均具有较强的适用性与可解释性。

本文提出的模型将最短路径搜索、动态规划、马尔可夫决策与蒙特卡洛模拟有机结合，可为复杂环境下的资源受限路径优化与群体协同决策问题提供参考。

关键词：Dijkstra 算法；动态规划；马尔可夫决策；蒙特卡洛模拟；路径优化；资源配置

一、 问题重述

1.1 问题背景

“穿越沙漠”是一个以资源受限条件下路径规划与决策优化为核心的策略类游戏。玩家需要在给定地图上，从起点出发，穿越沙漠并最终到达终点。游戏过程中，玩家必须合理安排每日行动、购买和消耗水与食物，并结合天气变化、村庄补给和矿山挖矿等机制制定最优策略，以保证在规定期限内完成穿越任务，同时尽可能保留更多资金。具体规则如下：

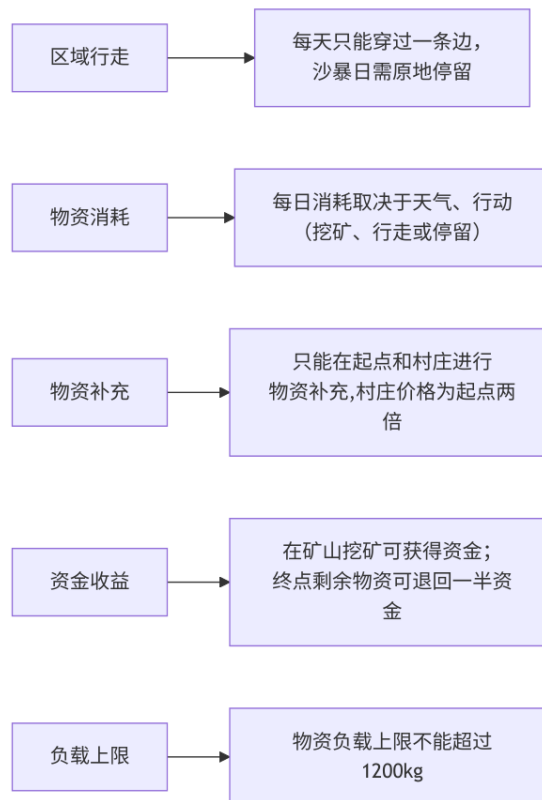


图 1: 问题背景

此外，在多名玩家的情况下，还存在同行物资消耗翻倍、多人同时挖矿收益压缩等规则。

1.2 需解决的问题

根据题目设定，需要针对不同信息条件和参与人数建立相应模型，为玩家制定最优或近优策略，使其在成功到达终点的前提下剩余资金尽可能多。具体需要解决如下问题：

- 问题一：单人、天气完全已知情形。假设仅有一名玩家参与游戏，且在整个游戏周期内每天的天气状况均已提前知道。需要在已知天气序列、地图结构、资源价格、

负重上限和截止日期的前提下，确定玩家每天的最优行动方案，包括是否移动、是否停留、是否前往村庄补给、是否前往矿山挖矿，以及初始阶段应购入多少水和食物，并对附件中的前两关进行求解。

- 问题二：单人、天气部分未知情形。假设仍只有一名玩家，但玩家仅知道当天的天气，而无法提前获知未来天气状况，只能根据当前状态决定当天行动。此时需要建立适用于随机天气环境下的动态决策模型，使玩家能够依据当前位置、剩余资源、剩余资金和剩余时间实时调整策略，并对附件中的第三关和第四关进行讨论。
- 问题三第一问：多人、天气已知情形。当有多名玩家同时从起点出发，且天气信息完全已知时，每位玩家的行动方案需在第 1 天确定后不再改变。由于多人同行、同矿和同村补给会引起资源消耗、收益和价格变化，因此需要建立多人协同优化模型，确定各玩家应采取的路径与分工策略，并对附件中的第五关进行具体分析。
- 问题三第二问：多人、天气部分未知情形。假设多名玩家仅知道当天的天气状况，并且从第 2 天起，所有玩家在每天结束后都可以获知其他玩家当日的行动方案及剩余资源情况，然后再决定下一天的行动。这一情形下，玩家之间既存在竞争，也存在一定的信息互动与协同关系。需要建立适用于动态博弈环境的决策模型，研究一般情况下玩家应采取的策略，并对附件中的第六关进行讨论。

二、 问题分析

2.1 问题一分析

在问题一中，玩家需要经过两个关卡，但是其本质都是一致的。玩家需要在遵守游戏规则的前提下，尽可能地多保留资金。于是问题一即为根据参与者在游戏过程中所要满足的各种条件前提下，如何最大化关于资金的函数。我们根据游戏规则，可以依据向前行走、沙暴时原地停留、矿山挖矿、村庄补给等情况，建立关于物资，资金和负重的递推表达式，最后可以化为单目标优化问题。考虑到题目图结构复杂，暴力搜索计算耗时较长，所以我们使用动态规划来求解。

2.2 问题二分析

第二问中未来天气未知，题目由确定性优化转化为随机动态决策问题，因此需要在状态转移中引入天气的不确定性，用马尔可夫决策思想描述“当前状态—行动选择—未来状态”的演化过程，使玩家能够依据当天天气和当前资源状况滚动调整策略。

2.3 问题三分析

第三问进一步加入多名玩家同时行动时的同行、同矿、同购惩罚，使问题由单人最优扩展为多主体耦合优化问题，不再只追求单条最优路径，而要综合考虑玩家之间的分工、错峰和协同；同时，由于天气过程具有随机性，本文采用蒙特卡洛方法模拟多种天气样本，对多人策略在不同天气实现下的收益与稳定性进行评估。

第五关在天气已知的前提下，分析挖矿收益与需要多消耗物资的关系，确定挖矿亏损，选择直达终点。在次基础上再考虑多人同行惩罚，采用一人停留来换取团队最大总收入，采用合作而非竞争的博弈策略。

第六关对于天气未知情况的三人决策，由于存在同行消耗惩罚和收益惩罚，故也采用合作，让一人挖矿，两人直达的方案进行。通过对天气情况与消耗的分析，确定是否需要补给以及补给次数，最后给出一般情况下的最优路径。

三、 模型假设

1. 不考虑途中资源损耗、丢失及额外获取等特殊情况；
2. 导致游戏无法进行的极端沙暴天气不会出现；
3. 除题目说明的价格变化外，不考虑其他额外成本；
4. 在蒙特卡洛模拟中，各次天气样本相互独立，可视为对真实天气过程的合理近似。

四、 符号说明

符号	含义
W	收益
$\max M$	负载上限
C_w	水基础耗量
C_f	食物基础耗量
P_w	水基准价格
P_f	食物基准价格
m_w	水质量
m_f	食物质量

续下页

符号	含义
G	剩余金额
B_w	水购买量
B_f	食物购买量
N_t	第 t 日所在区域编号
$\Omega_s = \{N_s\}$	起点编号集合
$\Omega_e = \{N_e\}$	终点编号集合
$\Omega_v = \{N_{v_i} \mid i \in \mathbb{N}^*\}$	村庄编号集合
$\Omega_h = \{N_{h_i} \mid i \in \mathbb{N}^*\}$	矿山编号集合
M	负重
F	食物剩余量
T	水剩余量
D	天气（1-晴朗，2-高温，3-沙暴）
X	动向（1-移动，2-停留，3-挖矿）
$T = \max t$	到终点时的天数
n	玩家总数
$i \in \{1, 2, \dots, n\}$	玩家编号
E	地图邻接边集合
a_{uv}	邻接矩阵元素，若区域 u, v 相邻则 $a_{uv} = 1$ ，否则 $a_{uv} = 0$
$Y_{i,t}^{uv}$	第 i 名玩家在第 t 日是否沿边 (u, v) 移动的 0-1 变量
$Z_{i,t}^h$	第 i 名玩家在第 t 日是否在矿山 $h \in \Omega_h$ 挖矿的 0-1 变量
$Q_{i,t}^v$	第 i 名玩家在第 t 日是否在村庄 $v \in \Omega_v$ 购买资源的 0-1 变量
k_t^{uv}	第 t 日沿边 (u, v) 同时移动的玩家数
k_t^h	第 t 日在矿山 h 同时挖矿的玩家数
k_t^v	第 t 日在村庄 v 同时购买资源的玩家数
δ	联盟稳定性指标

续下页

符号	含义
$\mathcal{P}_i$	由前两问路径模型生成的第 i 名玩家候选可行路径集
$\pi_t(d' d)$	天气状态从 d 转移到 d' 的条件概率
$V_t(\cdot)$	第 t 日滚动决策价值函数

注：未列出符号及重复的符号以出现处为准

五、模型的建立与求解

5.1 问题 1 的建模和求解

根据题意，玩家停留和移动都需要消耗物资，可在起点和村庄购买物资，在矿山处可以挖矿赚取资金，但沙尘暴天气无法前进，最后可以在终点处退还剩余的物资换取资金。玩家需要在 30 天内从起点出发，最终到达终点，要求最大化最终的金额。

为刻画题目中各区域之间的通行关系，本文首先引入 Dijkstra 最短路径算法对地图进行预处理。Dijkstra 算法是一种用于求解非负权图单源最短路径问题的经典算法，适用于边权均为非负的网络结构。将地图抽象为无向赋权图

$$G = (V, E),$$

其中， V 表示地图中所有区域节点的集合， E 表示节点之间可直接到达的边集合。若节点 v_i 与节点 v_j 在地图中相邻，则记 $(v_i, v_j) \in E$ 。进一步设边权函数为

$$w : E \rightarrow \mathbb{R}_{\geq 0},$$

其中， $w(v_i, v_j)$ 表示从节点 v_i 移动到节点 v_j 所需的代价。由于本题中玩家每天最多只能移动到一个相邻区域，且单次移动不存在负代价，故可将相邻区域之间的边权统一记为 1，从而将原问题转化为标准的非负权最短路问题。

Dijkstra 算法的核心思想是“逐步逼近最优解”。设起点为 s ，定义 $d(v)$ 为起点 s 到节点 v 的当前最短距离估计值，初始化为

$$d(s) = 0, \quad d(v) = +\infty \quad (v \neq s).$$

随后，在每一步迭代中，从未确定最短距离的节点中选取具有最小 $d(v)$ 的节点，将其加入已确定集合，并对其所有邻接节点进行松弛操作：

$$d(v_j) = \min\{d(v_j), d(v_i) + w(v_i, v_j)\}.$$

重复上述过程，直至所有节点的最短距离均被确定。由于该算法始终在非负权条件下扩展当前最优路径，因此能够保证所得结果即为全局最短路径。该方法不仅可以得到起点到任意节点的最短距离，还可以通过记录前驱节点恢复完整路径序列。

在本题中，Dijkstra 算法主要用于求解起点、村庄、矿山和终点等关键位置之间的最短路径及对应距离，为后续优化模型提供基础数据支持。具体而言，先利用该算法分别计算各关键节点对之间的最短通行路线，得到路径长度与节点经过顺序；再将最短路径结果嵌入后续决策模型中，与天气状态、水和食物消耗、负重约束、补给策略及挖矿收益等因素相结合，进一步建立收益最大化模型。这样处理能够有效减少复杂地图上的无效路径枚举，将“路径搜索”与“资源决策”分层求解，从而提高模型求解效率，并增强整体建模过程的条理性与可解释性。

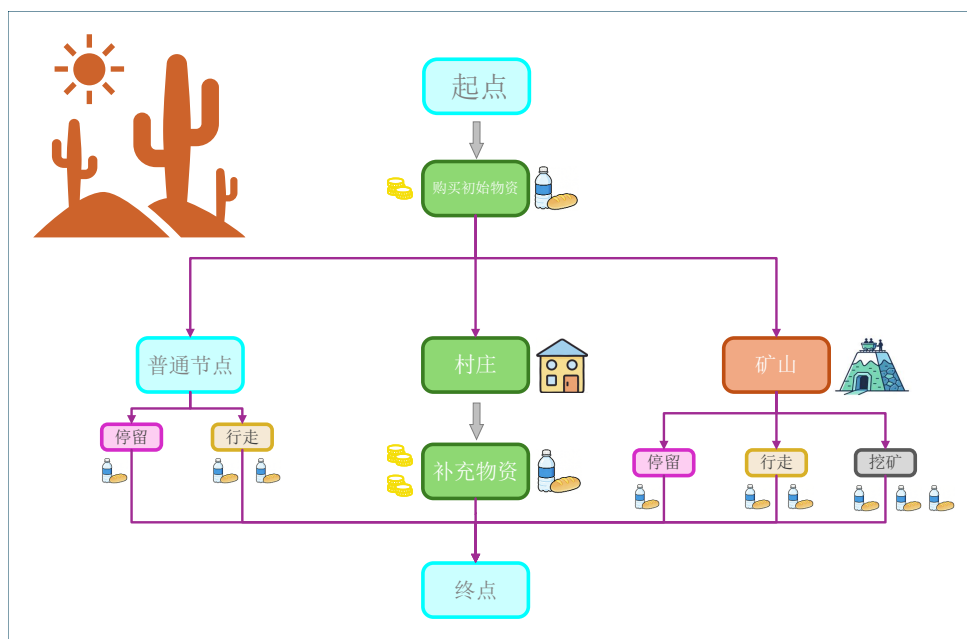


图 2: 题目示意图

5.1.1 最佳策略

为求解每天天气状况事先全部已知情况下的最佳策略：

- 在不考虑沙尘暴以及经过矿山的情况下，玩家只沿最短路径走并且中间不停留。
- 在沙尘暴天气下，玩家在矿山等待或挖矿。
- 玩家在起点处购买尽可能多的物资，且只在村庄购买最低需求的物资。

第一点，在不考虑沙尘暴以及经过矿山的情况下，玩家无法在中途补充金额，因此消耗越少最后剩余的金额越多。而原地停留只会增加物资的消耗，即资金的损耗，因此显然地，玩家必须沿着最短路径走。

第二点，在沙尘暴天气下，玩家无法前进，又因为此时挖矿能增加金额，因此在矿山等待或挖矿。

第三点，由于起点处的物资价格较低，因此玩家应该在起点处购买尽可能多的物资可以最低限度地减少资金的损耗。但是起点购买的物资可能不足以支持整个游戏过程，因此需要在村庄进行补给，同样需要减少资金的损耗，即购买尽可能少的物资。

5.1.2 模型

则问题一可整理为如下确定性动态优化模型：

$$\begin{aligned} \max \quad & S_1 = G_\tau + \frac{1}{2}P_w T_\tau + \frac{1}{2}P_f F_\tau \\ \text{s.t.} \quad & \begin{cases} N_0 = N_s, & N_\tau = N_e, & 0 \leq \tau \leq T_{\max}, \\ T_t = T_{t-1} + B_t^w - c_w(D_t, a_t), & t = 1, 2, \dots, \tau, \\ F_t = F_{t-1} + B_t^f - c_f(D_t, a_t), & t = 1, 2, \dots, \tau, \\ G_t = G_{t-1} - p_t^w B_t^w - p_t^f B_t^f + r(a_t), & t = 1, 2, \dots, \tau, \\ m_w T_t + m_f F_t \leq M_{\max}, & t = 0, 1, \dots, \tau, \\ T_t \geq 0, \quad F_t \geq 0, \quad G_t \geq 0, & t = 0, 1, \dots, \tau, \\ a_t = \text{move} \Rightarrow (N_{t-1}, N_t) \in \mathcal{E}, \\ a_t = \text{stay} \Rightarrow N_t = N_{t-1}, \\ a_t = \text{mine} \Rightarrow N_{t-1} \in \Omega_h, \quad N_t = N_{t-1}, \\ D_t = 3 \Rightarrow a_t \neq \text{move}, \\ B_t^w, B_t^f \geq 0, & t = 0, 1, \dots, \tau, \\ B_t^w = B_t^f = 0, & N_{t-1} \notin \{N_s\} \cup \Omega_v, \quad t \geq 1. \end{cases} \end{aligned} \quad (1)$$

其中，

$$c_w(D_t, a_t) = \begin{cases} 2\hat{c}_w(D_t), & a_t = \text{move}, \\ \hat{c}_w(D_t), & a_t = \text{stay}, \\ 3\hat{c}_w(D_t), & a_t = \text{mine}, \end{cases} \quad c_f(D_t, a_t) = \begin{cases} 2\hat{c}_f(D_t), & a_t = \text{move}, \\ \hat{c}_f(D_t), & a_t = \text{stay}, \\ 3\hat{c}_f(D_t), & a_t = \text{mine}, \end{cases}$$

$$r(a_t) = \begin{cases} R_0, & a_t = \text{mine}, \\ 0, & \text{otherwise}, \end{cases} \quad p_t^w = \begin{cases} P_w, & t = 0, \quad N_0 = N_s, \\ 2P_w, & N_{t-1} \in \Omega_v, \\ 0, & \text{otherwise}, \end{cases}$$

$$p_t^f = \begin{cases} P_f, & t = 0, N_0 = N_s, \\ 2P_f, & N_{t-1} \in \Omega_v, \\ 0, & \text{otherwise.} \end{cases}$$

对于最短路径，我们采用最经典的 Dijkstra 算法进行求解 [4]。其通过按路径长度递增的顺序，在非负权重的图中逐个确定从源点到所有其他顶点的最短路径，并利用松弛操作不断更新当前的最优距离。

利用 C++ 编程，我们求解出了第一关和第二关的最优策略和最优收益，结果如下：

关卡	最优收益
第一关	10470
第二关	12730

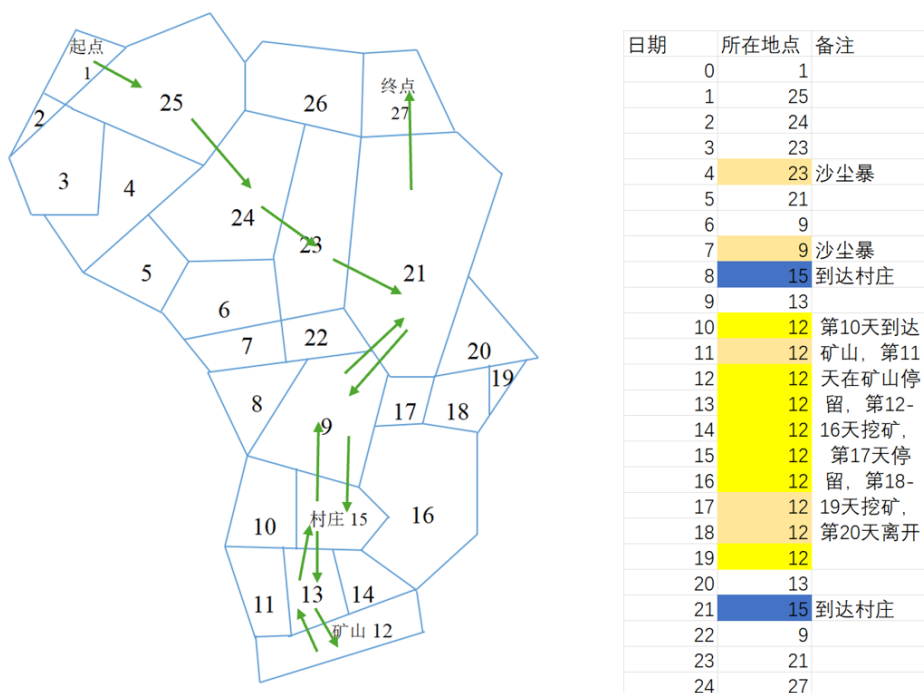


图 3: 问题一第一关结果图

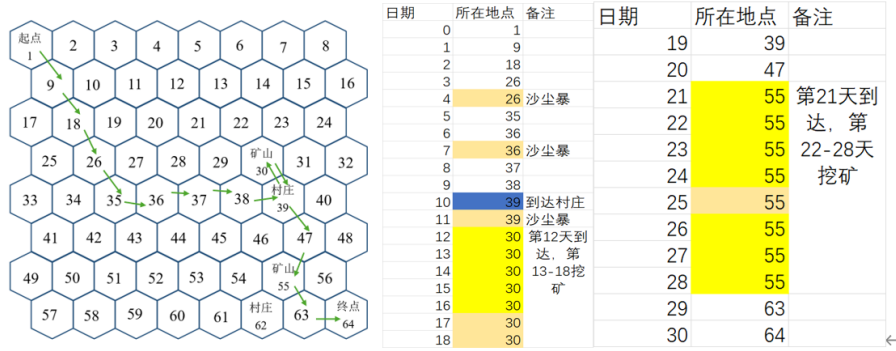


图 4: 问题一第二关结果图

5.2 问题 2 的建模和求解

5.2.1 模型的建立

1. 马尔可夫性质的引入

在本题中, 因为玩家仅知道当天的天气状况, 并需要据此决定当天的行动方案, 所以过去的天气对当下已经没有任何参考价值。那么天气随机的限制可简化为在每一天中只知道当前的天气状态, 同时通过预测下一天的天气来帮助玩家进行下一步的决策, 根据该天气性质, 可以引入马尔可夫性来解决此问题: 即下一状态只依赖于当前状态, 与过去状态无关。以下给出马尔可夫的数学表达式:

$$P(X_{n+1} = x \mid X_1 = x_1, X_2 = x_2, \dots, X_n = x_n) = P(X_{n+1} = x \mid X_n = x_n) \quad (2)$$

2. 引入马尔可夫后的决策过程

在玩家决策时, 不仅要考虑天气的外部因素, 还需要考虑自身负重、行走天数、水和食物等内部因素, 不同的未来天气会让玩家做出不同的决策。所以要建立基于马尔可夫性质的决策方程:

$$F = (D_t, I, G_t) \quad (3)$$

其中, D_t 为第 t 天的天气, I 为内部因素 (包括自身负重、行走天数、水和食物), G_t 为在 D 这个天气序列下在第 t 天的剩余资金。

3. 约束条件的界定

在题目中, 首先要保证在规定的时间内到达终点, 其次才是保留尽可能多的资金。所以我们应该优先考虑能否走到终点。即我们应该在起点买足所有的水和食物, 以确保在最坏的天气情况下可以到达村庄或终点来补充物资:

$$T_t \geq 0, F_t \geq 0 \quad (4)$$

其中, S 为从起点到村庄或终点的路径。

4. 具有马尔可夫性质的模型总结

其中终端条件为问题二标准约束形式

$$V_\tau(S_\tau) = G_\tau + \frac{1}{2}P_w T_\tau + \frac{1}{2}P_f F_\tau, \quad N_\tau = N_e. \quad (5)$$

$$\text{s.t.} \left\{ \begin{array}{l} S_0 = (N_s, T_0, F_0, G_0, D_0, 0), \\ T_{t+1} = T_t + B_t^w - c_w(D_t, a_t), \\ F_{t+1} = F_t + B_t^f - c_f(D_t, a_t), \\ G_{t+1} = G_t - p_t^w B_t^w - p_t^f B_t^f + r(a_t), \\ m_w T_{t+1} + m_f F_{t+1} \leq M_{\max}, \\ T_{t+1} \geq 0, \quad F_{t+1} \geq 0, \quad G_{t+1} \geq 0, \\ a_t = \text{move} \Rightarrow (N_t, N_{t+1}) \in \mathcal{E}, \\ a_t = \text{stay} \Rightarrow N_{t+1} = N_t, \\ a_t = \text{mine} \Rightarrow N_t \in \Omega_h, \quad N_{t+1} = N_t, \\ D_t = 3 \Rightarrow a_t \neq \text{move}, \\ B_t^w = B_t^f = 0, \quad N_t \notin \{N_s\} \cup \Omega_v, \quad t \geq 1, \\ 0 \leq t \leq T_{\max} - 1. \end{array} \right.$$

其中单步回报函数可统一记为

$$R_t(S_t, a_t, B_t^w, B_t^f) = -p_t^w B_t^w - p_t^f B_t^f + r(a_t).$$

5.2.2 模型的求解及结果

动态规划求解

1. 数据初始化根据表格数据初始化各天气的食物和水的消耗和路线, 设定每个状态的最大收益为无穷小。同时根据第一天的天气给出初始水和食物的购买方案。

2. 约束条件的补充

动态规划算法必须当前一步有解才能进行下一步的状态转移, 所以我们要确定在不同天气序列下的解的范围。

在每个状态下, 都先从最坏的情况下进行考虑, 假设后面全部都是高温, 从而获得一个能走到终点的结果, 即所有解的下界。

3. 动态规划的求解

利用跟问题一相同的动态规划算法求解。

4. 结果线路的输出

利用 dfs 回溯法来反向寻找经过路线。

根据以上流程来进行第三关，第四关的求解。

结果呈现和分析

1. 第三关结果呈现

由于第三关仅有晴朗和高温两种天气情况，所以我们采用二叉决策树来表现在每一个点上的决策和状态变化。结果如下图。

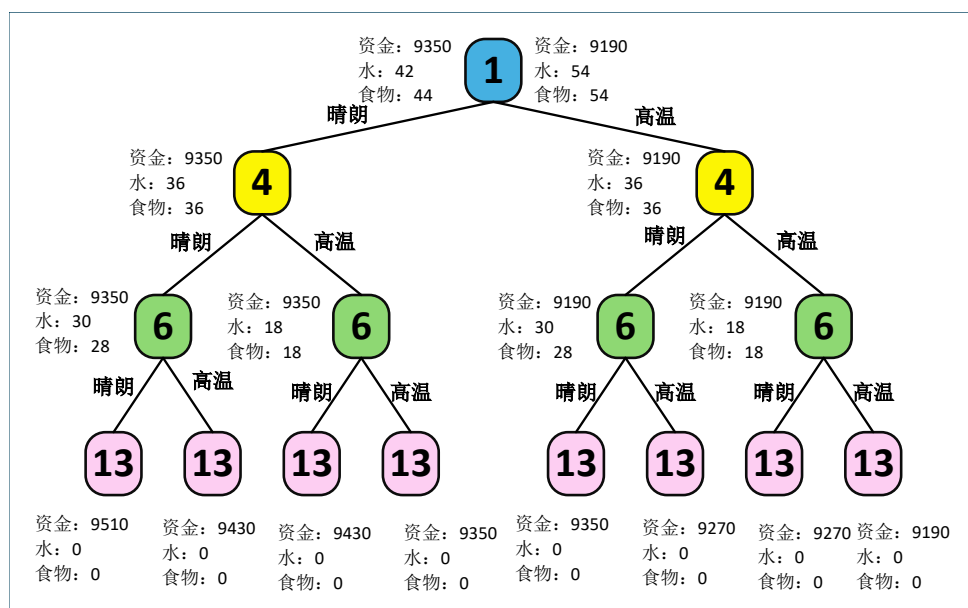


图 5: 第三关结果图

我们可以发现路线均是从 $1 \rightarrow 4 \rightarrow 6 \rightarrow 13$ 。说明在该沙漠的地图下，通过如上的决策得到的最优解相同，所以该决策具有稳定性。

2. 第四关结果呈现

由于第四关存在晴朗，高温和沙暴三种天气情况，且到终点的最少天数也需要 10 天，所以使用二叉决策树会使得结果冗余。故采用画图法来展示结果

与第三关相同在每个状态下，都先从最坏的情况下进行考虑，假设后面全部都是高温，中间小概率随机插入沙暴天气，从而获得一个能走到终点的结果，即所有解的下界

如下是其中一种可能的路线结果：

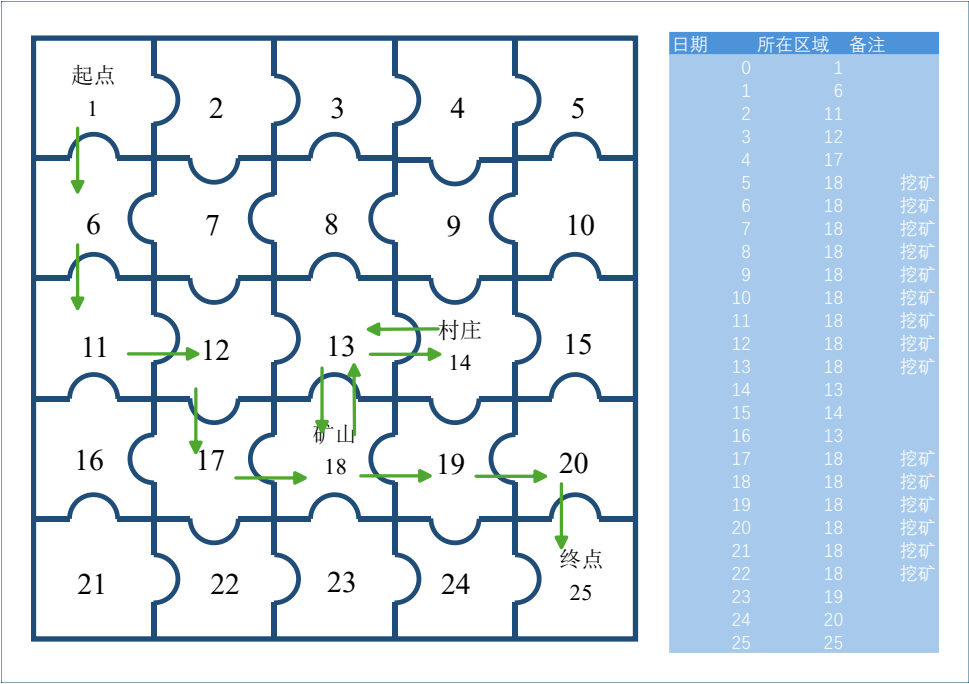


图 6: 第四关结果图

所有时刻的各种状态如下：

天数	当前位置	当前水	当前食物	利润	天数	当前位置	当前水	当前食物	利润
0	1	196	305	5970	13	18	40	124	13970
1	6	190	297	5970	14	13	22	106	14970
2	11	184	289	5970	15	14	225	225	15970
3	12	178	281	5970	16	13	219	217	16970
4	17	172	273	5970	17	18	213	209	17970
5	18	166	265	5970	18	18	186	182	17970
6	18	157	253	6970	19	18	159	155	17710
7	18	130	226	7970	20	18	132	128	16860
8	18	103	199	8970	21	18	105	101	16860
9	18	94	187	9970	22	18	96	89	17860
10	18	85	175	10970	23	19	90	81	17860
11	18	76	163	11970	24	20	84	73	17860
12	18	67	151	12970	25	25	78	65	17860

5.3 问题 3 的建模和求解

5.3.1 总述

前两问分别讨论了单人情形下“已知全程天气”的静态最优策略与“仅知当天天气”的动态最优策略。第三问在此基础上引入多名玩家同时出发的协同决策机制，并在题设中额外加入三类拥挤效应：

- 同行惩罚：若某日有多名玩家沿同一边同时移动，则每位玩家的移动消耗按更高倍率计算；
- 同矿惩罚：若某日有多名玩家在同一矿山挖矿，则每位玩家的挖矿消耗更高且单位收益下降；
- 同购惩罚：若某日有多名玩家在同一村庄同时购买资源，则每箱资源价格上升。

因此，第三问已不再是单纯的“最优路径”问题，而是路径选择、出发时机、停留时机、补给时机与挖矿时机的多主体耦合优化问题。本文将前两问的路径模型作为第三问的基础模块，建立“最短路预处理 + 单人可行策略集生成 + 多人协同调度优化 + 未知天气滚动博弈修正”的统一建模框架 [3, 2]。

我们可将问题分为两类情形：

- 每天气象序列事先已知，各玩家第 0 天即需确定全程方案，属于静态协同规划问题；
- 仅知道当天天气，且每天结束后可观察其他玩家状态并修正次日决策，属于动态协同博弈问题。

为统一处理这两类问题，本文将第三问拆解为三个层次 [1]：

- 路径层：用前两问中“最短路 + 资源可行性判定”模型，生成每位玩家从起点到终点、起点到矿山、村庄到矿山、矿山到终点等若干最短或次短候选路径，并形成路径库 $\mathcal{P}_i$ 。
- 调度层：在候选路径库基础上，对不同玩家的出发日期、等待日期、补给日期、挖矿日期进行联合优化，以降低拥挤效应造成的额外损失。
- 博弈层：在未知天气条件下，结合天气转移概率、剩余资源和他人状态，采用滚动时域优化思想动态重规划路径与角色分工。

这一本质上是从单人“点到点最优”推广到多人“时空网络上的协调最优”。

5.3.2 多人资源演化方程

第 i 名玩家在第 t 日结束后对应负重为

$$M_{i,t} = m_w T_{i,t} + m_f F_{i,t} \leq \max M.$$

为统一描述三种天气和三种动作，引入日资源消耗系数、日价格系数和日盈利系数

$$\alpha(D_t, X_{i,t}, \mathbf{k}_t), \quad \beta(D_t, X_{i,t}, \mathbf{k}_t), \quad \gamma(D_t, X_{i,t}, \mathbf{k}_t).$$

分别表示第 t 日水与食物的消耗倍率，其中 $\mathbf{k}_t$ 表示第 t 日的拥挤状态（有多少玩家一起行动）。于是资源递推方程可写为

$$T_{i,t} = T_{i,t-1} + B_{w,i}(t) - \beta(D_t, X_{i,t}, \mathbf{k}_t)C_w, F_{i,t} = F_{i,t-1} + B_{f,i}(t) - \beta(D_t, X_{i,t}, \mathbf{k}_t)C_f.$$

其中， $B_{w,i}(t), B_{f,i}(t)$ 仅当玩家在起点初始采购或在村庄补给。若玩家未到终点前出现

$$T_{i,t} < 0 \quad \text{或} \quad F_{i,t} < 0,$$

则判定该策略失效。

资金递推方程为

$$G_{i,t} = G_{i,t-1} - \Pi_{i,t}^{\text{buy}} + \Pi_{i,t}^{\text{mine}},$$

其中 $\Pi_{i,t}^{\text{buy}}$ 为第 t 日购买资源的支出， $\Pi_{i,t}^{\text{mine}}$ 为第 t 日挖矿收益。到达终点后，剩余资源按半价回收，因此玩家最终收益记为

$$S_i = G_{i,T} + \frac{1}{2}P_w T_{i,T} + \frac{1}{2}P_f F_{i,T}.$$

联盟总收益为

$$S = \sum_{i=1}^n S_i.$$

5.3.3 拥挤效应的定量刻画

第三问区别于前两问的关键，在于玩家间相互作用改变了消耗、收益与价格。为此分别刻画三类拥挤效应。

同行惩罚 移动状态下的资源消耗可统一写为

$$\beta(D_t, 1, \mathbf{k}_t) = 2k_t^{uv} \beta_0(D_t, 1),$$

其中 β_0 为单人情况下由天气和动作确定的基础消耗倍率。

同矿惩罚 根据题意，若多人同矿挖矿，则每人的资源消耗放大，且每人每天获得收益降为基础收益的某一分配比例。对应的消耗倍率写成

$$\beta(D_t, 3, \mathbf{k}_t) = 3 \times \beta_0(D_t, 3).$$

对应获得的资金倍率以及资金为

$$\gamma(D_t, 3, \mathbf{k}_t) = \frac{1}{k_t^h} \times \gamma_0(D_t, 3), \Pi_{i,t}^{\text{mine}} = \gamma(D_t, 3, \mathbf{k}_t) \times W_0.$$

W_0 为单人挖矿基础收益。当 k_t^h 较大时，虽然联盟总体仍可能因多点挖矿而受益，但单个矿点的边际收益会显著下降，因此“矿山分流”成为多人策略中的核心。

同购惩罚 令

$$k_t^v = \sum_{i=1}^n Q_{i,t}^v$$

表示第 t 日在村庄 v 同时购买资源的玩家数。于是购买成本写为

$$\Pi_{i,t}^{\text{buy}} = 4 \times [P_w B_{w,i}(t) + P_f B_{f,i}(t)].$$

该式体现出“同步补给越集中，补给成本越高”，因此在多人情形下应尽量错开采购时间，或将玩家分散到不同补给窗口。

联盟目标函数：收益与稳定性的双目标规划 若仅以联盟总收益 G_S 为目标，则可能出现“少数玩家获利很高、其余玩家收益很低”的情况；若只追求均分收益，则又可能牺牲整体最优。所以我们确定第三问为收益-稳定性双目标优化问题。

定义联盟稳定性为

$$\delta = 1 - \frac{\sqrt{\frac{1}{n} \sum_{i=1}^n (G_i - \bar{G})^2}}{\bar{G} + \varepsilon},$$

其中 $\varepsilon > 0$ 为防止分母为零的极小量。显然， δ 越接近 1，表示各成员收益越均衡，联盟越稳定。

于是第三问的双目标模型可表述为

$$\max G_S = \sum_{i=1}^n G_i, \max \delta.$$

考虑实际求解时需将双目标转化为单目标，采用加权求和形式：

$$\max J = \lambda \frac{G_S}{G_{S,r}} + (1 - \lambda)\delta, \quad 0 \leq \lambda \leq 1.$$

其中 $G_{S,r}$ 为单人模型或独立决策模型给出的参考收益上界。 λ 为权重。若更强调联盟整体利润，可适当提高 λ ；若更强调协作稳定性，则减小 λ 。

于是问题三（1）的标准模型可写为

$$\max J = \lambda \frac{S_{\text{sum}}}{S_{\text{ref}}} + (1 - \lambda)\delta, \quad 0 \leq \lambda \leq 1. \quad (6)$$

$$\text{s.t.} \begin{cases} T_{i,t} = T_{i,t-1} + B_{i,t}^w - c_{i,t}^w, & i = 1, \dots, n, \ t = 1, \dots, \tau_i, \\ F_{i,t} = F_{i,t-1} + B_{i,t}^f - c_{i,t}^f, & i = 1, \dots, n, \ t = 1, \dots, \tau_i, \\ G_{i,t} = G_{i,t-1} - \Pi_{i,t}^{\text{buy}} + \Pi_{i,t}^{\text{mine}}, & i = 1, \dots, n, \ t = 1, \dots, \tau_i, \\ m_w T_{i,t} + m_f F_{i,t} \leq M_{\max}, & i = 1, \dots, n, \ t = 0, \dots, \tau_i, \\ T_{i,t} \geq 0, \quad F_{i,t} \geq 0, \quad G_{i,t} \geq 0, & i = 1, \dots, n, \ t = 0, \dots, \tau_i, \\ N_{i,0} = N_s, \quad N_{i,\tau_i} = N_e, \quad 0 \leq \tau_i \leq T_{\max}, \\ a_{i,t} = \text{move} \Rightarrow (N_{i,t-1}, N_{i,t}) \in \mathcal{E}, \\ a_{i,t} = \text{stay} \Rightarrow N_{i,t} = N_{i,t-1}, \\ a_{i,t} = \text{mine} \Rightarrow N_{i,t-1} \in \Omega_h, \ N_{i,t} = N_{i,t-1}, \\ D_t = 3 \Rightarrow a_{i,t} \neq \text{move}, \quad i = 1, \dots, n, \\ B_{i,t}^w = B_{i,t}^f = 0, \quad N_{i,t-1} \notin \{N_s\} \cup \Omega_v, \ t \geq 1. \end{cases}$$

其中

$$c_{i,t}^w = \begin{cases} \rho^{\text{move}}(k_{uv,t}) \cdot 2\hat{c}_w(D_t), & a_{i,t} = \text{move}, \\ \hat{c}_w(D_t), & a_{i,t} = \text{stay}, \\ \rho^{\text{mine}}(k_{h,t}) \cdot 3\hat{c}_w(D_t), & a_{i,t} = \text{mine}, \end{cases}$$

$$c_{i,t}^f = \begin{cases} \rho^{\text{move}}(k_{uv,t}) \cdot 2\hat{c}_f(D_t), & a_{i,t} = \text{move}, \\ \hat{c}_f(D_t), & a_{i,t} = \text{stay}, \\ \rho^{\text{mine}}(k_{h,t}) \cdot 3\hat{c}_f(D_t), & a_{i,t} = \text{mine}, \end{cases}$$

$$\Pi_{i,t}^{\text{buy}} = \rho^{\text{buy}}(k_{v,t}) \left(P_w B_{i,t}^w + P_f B_{i,t}^f \right),$$

$$\Pi_{i,t}^{\text{mine}} = \begin{cases} \frac{R_0}{k_{h,t}}, & a_{i,t} = \text{mine}, \\ 0, & \text{otherwise.} \end{cases}$$

这就是已知天气条件下的多人静态协同优化标准形式。

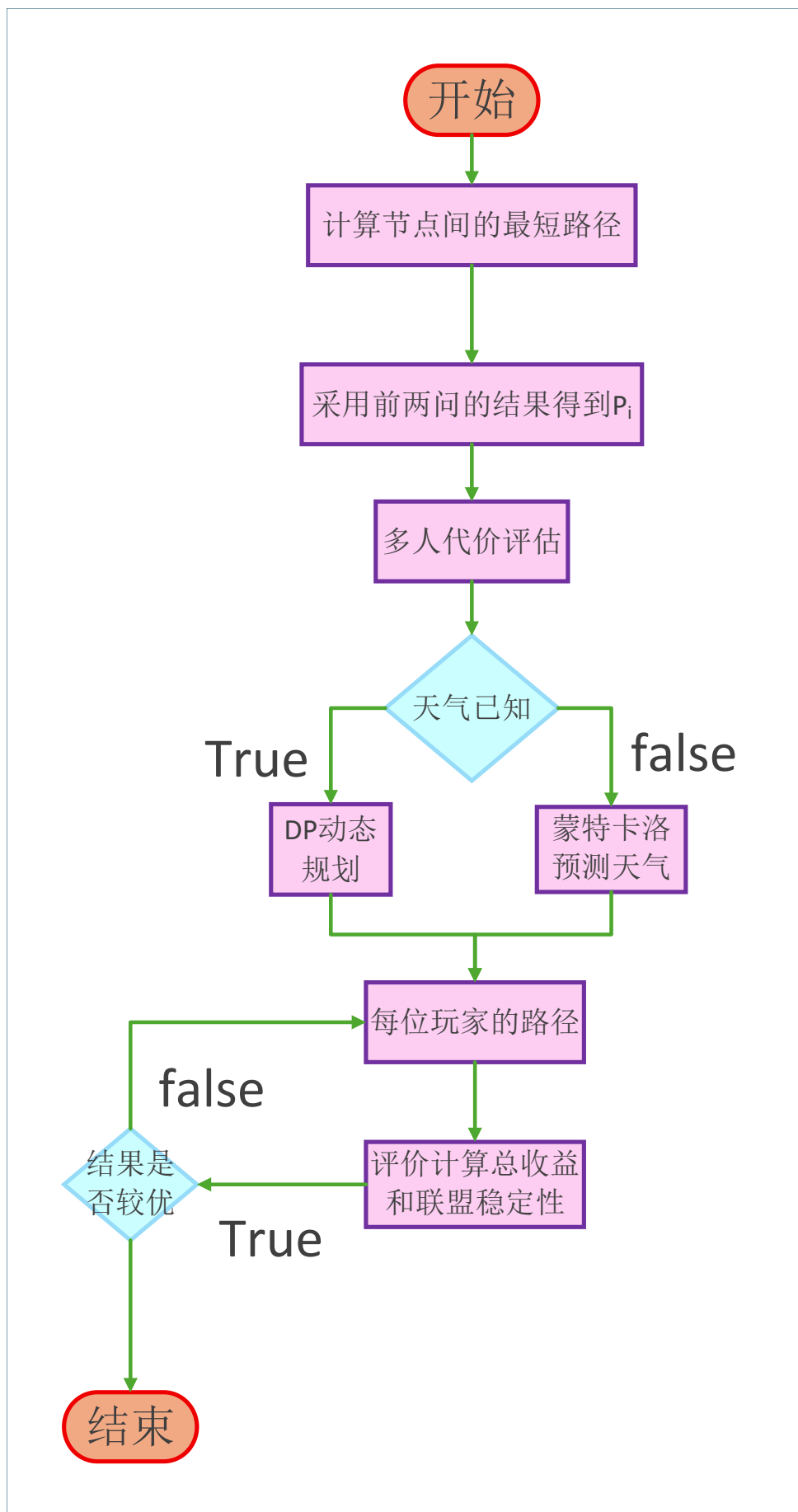


图 8: 第三题流程图

5.3.4 问题三第五关的分析和求解

由于第五关地图较为简单，我们直接进行分析和求解。

第五关没有村庄，所以无需考虑补给问题。现已知两特殊节点之间的最短路径（如图第五关示意图）。

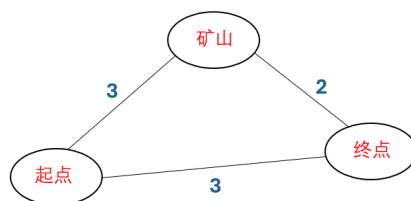


图 9: 第五关示意图

如果玩家决策为去矿山挖矿，那么考虑在矿山的消耗物资的价格为 $3 \times (3 \times 5 + 4 \times 10) = 165$ （晴朗）， $3 \times (9 \times 5 + 9 \times 10) = 405$ ，因此只有晴朗挖矿才可能盈利（最多盈利 35 元）。加上路上多消耗的天物资资金至少为 380，而根据天气情况挖矿最多赚 $2 \times 35 = 70$ ，显然无法实现通过挖矿盈利的目的。

因此，我们考虑直达终点的情况。直达终点有两条最短路径（ $1 \rightarrow 5 \rightarrow 6 \rightarrow 13$ 和 $1 \rightarrow 4 \rightarrow 6 \rightarrow 13$ ），一条备选次短路（ $1 \rightarrow 4 \rightarrow 6 \rightarrow 12 \rightarrow 13$ ）。由于两人一同运动时会消耗 4 倍基础消耗量，所以应考虑分流。我们分别计算一人走最短路 1，一人走次短路（如图第五关分开走示意图所示）和两人分别走两条最短路（如图第五关一起走示意图所示）的最终消耗。其中以蓝色路线为 A 玩家，橙色路线为 B 玩家。

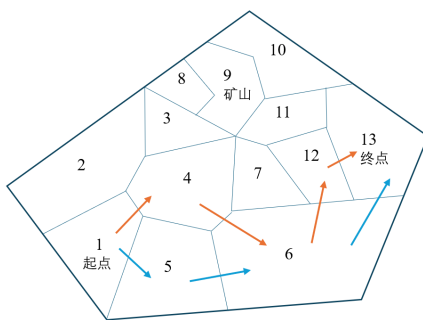


图 10: 第五关分开走示意图

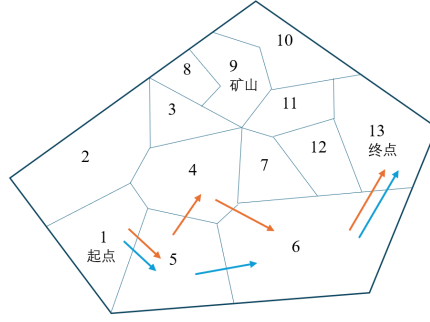


图 11: 第五关一起走示意图

由此可得

$$\begin{aligned} 490(A) + 600(B) &= 1090 && \text{(图第五关分开走示意图),} \\ 600(A = B) \times 2 &= 1200 && \text{(图第五关一起走示意图)} \end{aligned}$$

对于图 5 关一起走示意图方案进一步优化, 为避免第三天两人一起移动, 让 A 玩家在第二天选择停留, 即 (图第五关优化示意图)。

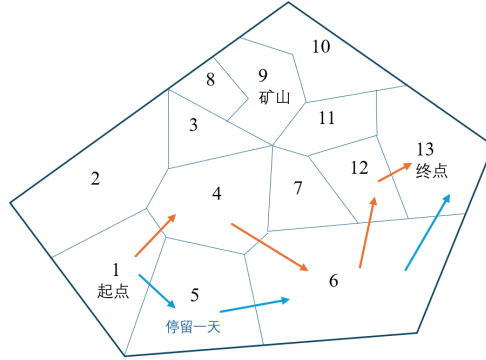


图 12: 第五关优化示意图

计算得到最优解

$$490(A) + 465(B) = 955$$

因此, 最终方案如第五关优化示意图所示。

5.3.5 第六关求解

第六关分析

天气演化模型 根据历史天气统计构造转移概率矩阵

$$\mathbf{P} = \begin{pmatrix} \pi(1|1) & \pi(2|1) & \pi(3|1) \\ \pi(1|2) & \pi(2|2) & \pi(3|2) \\ \pi(1|3) & \pi(2|3) & \pi(3|3) \end{pmatrix}.$$

则在第 t 日已知天气为 $D_t = d$ 时, 对未来第 $t + 1$ 日天气的预测分布为

$$\mathbb{P}(D_{t+1} = d' \mid D_t = d) = \pi(d' \mid d).$$

若需预测未来多日, 则可用联盟优先级协调蒙特卡洛链幕次转移

$$\mathbb{P}(D_{t+\tau} = d' \mid D_t = d) = (\mathbf{P}^\tau)_{dd'}.$$

这一步将第二问中“依据当天天气滚动修正”的思想进一步形式化。

状态变量与价值函数 记第 t 日联盟整体状态为

$$\mathcal{X}_t = \left(\{N_{i,t}\}_{i=1}^n, \{T_{i,t}\}_{i=1}^n, \{F_{i,t}\}_{i=1}^n, \{G_{i,t}\}_{i=1}^n, D_t \right).$$

则联盟在第 t 日的优化目标写为

$$V_t(\mathcal{X}_t) = \max_{\mathbf{u}_t} \{r_t(\mathcal{X}_t, \mathbf{u}_t) + \mathbb{E}[V_{t+1}(\mathcal{X}_{t+1}) \mid \mathcal{X}_t, \mathbf{u}_t]\},$$

其中 $\mathbf{u}_t$ 表示第 t 日所有玩家的联合动作, r_t 为当日即时回报, 包括挖矿收益、补给支出、资源回收价值变化及拥挤损失等。

由于直接求精确动态规划维数过高, 本文采用滚动时域近似: 每天仅向后预测 H 天, 在该短预测窗内求解近似最优联合动作; 次日获得新天气和新状态后, 再重新规划。这一方法兼顾了计算可行性与策略适应性。

联盟协同规则 在动态环境下, 多人最优不仅取决于路径, 还取决于角色分工。结合前两问可得, 玩家在第六关中通常会出现以下三类角色:

- 返程优先型: 剩余资源偏少且已接近终点, 优先采用最短路回终点;
- 挖矿收益型: 位于矿山附近且资源尚充足, 可继续挖矿获取额外收益;
- 补给缓冲型: 位于村庄附近, 用于承担补给和中转功能, 避免多人同日同矿或同购。

因此, 可为联盟制定如下优先级规则:

- 若某玩家的安全裕度

$$R_{i,t} = \frac{T_{i,t}}{\hat{T}_{i,t}^{\text{need}}} + \frac{F_{i,t}}{\hat{F}_{i,t}^{\text{need}}}$$

低于阈值 θ_1 , 则该玩家次日优先返程;

- 若某玩家处于矿山附近且预计未来若干日晴朗概率较高, 则优先挖矿;
- 若预计未来高温或沙暴概率升高, 则减少矿山驻留人数, 并提前向终点方向收缩;
- 若村庄将发生多人同购, 则对补给日期进行错峰, 控制 k_t^v 。

通过这些规则, 可把原本高度耦合的多人策略分解为“少数关键优先级 + 每日局部重规划”的形式。

第六关模型 第六关中仅知道玩家数目 $n=3$ ，且天气较少出现沙暴。基于问题二已知这个地图中选择挖矿可获取最大剩余资金。但由于多名玩家同时挖矿会导致挖矿利润骤减，因此可以考虑合作而非竞争的模式。即在满足 3 人游戏都成功的前提下，只看重集体盈利，个人亏损权重较小。

综合考虑，可得出以下策略：

- 让玩家 A 进行挖矿，玩家 B、C 选择直达终点（尽量避让同行）；
- 为了让三名玩家分流，所以规定挖矿者 A 走 $1 \rightarrow 6 \rightarrow 11 \rightarrow 16 \rightarrow 17 \rightarrow 18$ （矿山）的行走路线；
- 由于与 1 相接的只有 2、6 两个模块，因此让 B 玩家先出发，C 玩家原地停留 1 天后出发（假设当天非沙暴天），可选起点 $\rightarrow$ 终点最短路线行走

基于以上分析，我们通过联盟优先级协调蒙特卡洛算法形成天气预测模型随机模拟天气状况。

因为根据已知，我们知道这里较少出现沙暴，所以控制沙暴天数占总天数的比例 $\leq 1/3$ ，且对于任意连续 j （ j 大于等于 2）天内，沙暴天数比例 $\leq 1/2$ 的概率在 90% 以上。根据非必要不停留的原则，从起点到终点需要移动 8 天，基于以上假设，我们可以得出最坏的情况即移动的 8 天都是高温天，总共路上有 4-10 天沙暴天。玩家消耗资源重量为 920kg-1220kg，其中只有 10 天沙暴天会导致游戏失败，此出现概率大约为 1.14%（假设沙暴天数在剩余 10% 概率内平分）。

同理，从起点到村庄的最坏天气状况为 5 日高温，3-10 日沙暴天。按此计算最多消耗重量为 600-950kg 资源因此，玩家存活到达村庄的概率是 100%。

对于一部分玩家，出于心理因素，会在村庄进行物资补给，以保证游戏 100% 的成功率。但对于 $n=3$ 的小基数玩家，我们建议无需补给物资。

对于挖矿者，根据问题二的分析，由于挖矿产生消耗，物资不够的概率较大，因此考虑去村庄补给。去村庄需要消耗至少 4 天，因此若补给次数过多必然会使挖矿天数减少。对于一般情况，我们模拟了村庄补给次数与挖矿盈利（挖矿所得-采购资源所耗资金）以及挖矿天数的关系，（采用天气比例为沙暴：晴朗：高温 = 0.15：0.40：0.45）结果见去村庄补给次数与最大盈利的关系图和去村庄补给次数与最优挖矿天数的关系图（半数是从矿山挖矿后去村庄补给，并不返回矿山而直达终点）。

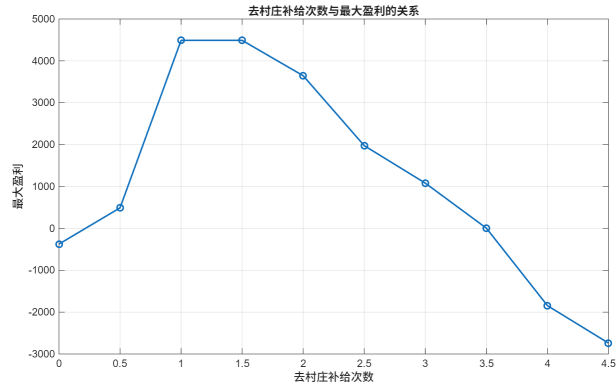


图 13: 去村庄补给次数与最大盈利的关系图

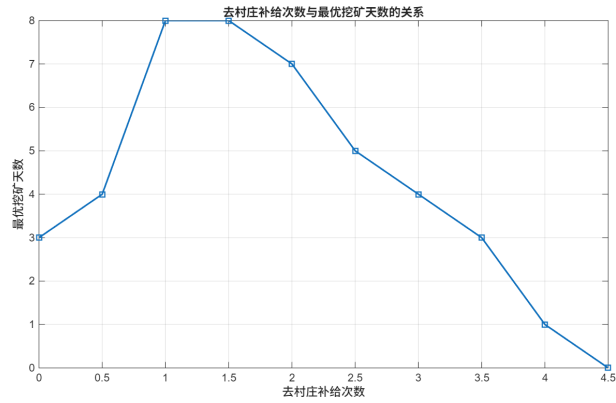


图 14: 去村庄补给次数与最优挖矿天数的关系图

由以上分析，可得出一般情况下的最优路线图（见图第六关一般情况下最优路线图）。额外说明：挖矿天数随沙暴比例的降低、晴朗比例的增加而增加，补给次数也相应可能变化但基本路线可参考以下路线图

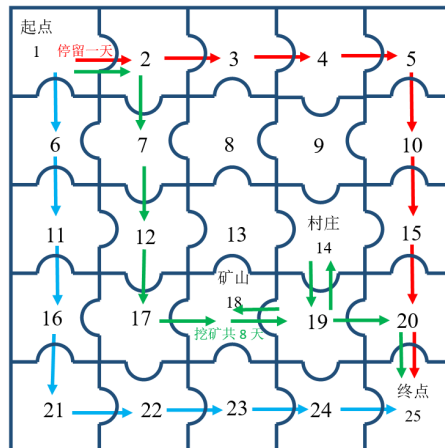


图 15: 第六关一般情况下最优路线图

为从第 t 天开始在状态 $\mathbf{X}_t$ 下能够获得的最大期望联盟效用，则问题三（2）可表述

为

$$V_t(\mathbf{X}_t) = \max_{\mathbf{u}_t, \{B_{i,t}^w, B_{i,t}^f\}_{i=1}^n} \left\{ R_t(\mathbf{X}_t, \mathbf{u}_t) + \sum_{d' \in \{1,2,3\}} \pi(d'|D_t) V_{t+1}(\mathbf{X}_{t+1}^{(d')}) \right\}, \quad (7)$$

终端条件为

$$V_\tau(\mathbf{X}_\tau) = \lambda \frac{\sum_{i=1}^n S_i}{S_{\text{ref}}} + (1 - \lambda)\delta. \quad (8)$$

其约束为

$$\text{s.t.} \begin{cases} T_{i,t+1} = T_{i,t} + B_{i,t}^w - c_{i,t}^w, & i = 1, \dots, n, \\ F_{i,t+1} = F_{i,t} + B_{i,t}^f - c_{i,t}^f, & i = 1, \dots, n, \\ G_{i,t+1} = G_{i,t} - \Pi_{i,t}^{\text{buy}} + \Pi_{i,t}^{\text{mine}}, & i = 1, \dots, n, \\ m_w T_{i,t+1} + m_f F_{i,t+1} \leq M_{\text{max}}, & i = 1, \dots, n, \\ T_{i,t+1} \geq 0, \quad F_{i,t+1} \geq 0, \quad G_{i,t+1} \geq 0, & i = 1, \dots, n, \\ a_{i,t} = \text{move} \Rightarrow (N_{i,t}, N_{i,t+1}) \in \mathcal{E}, \\ a_{i,t} = \text{stay} \Rightarrow N_{i,t+1} = N_{i,t}, \\ a_{i,t} = \text{mine} \Rightarrow N_{i,t} \in \Omega_h, \quad N_{i,t+1} = N_{i,t}, \\ D_t = 3 \Rightarrow a_{i,t} \neq \text{move}, & i = 1, \dots, n, \\ B_{i,t}^w = B_{i,t}^f = 0, \quad N_{i,t} \notin \{N_s\} \cup \Omega_v, \quad t \geq 1, \\ 0 \leq t \leq T_{\text{max}} - 1. \end{cases}$$

其中

$$R_t(\mathbf{X}_t, \mathbf{u}_t) = \lambda \frac{\sum_{i=1}^n (-\Pi_{i,t}^{\text{buy}} + \Pi_{i,t}^{\text{mine}})}{S_{\text{ref}}} + (1 - \lambda) \delta_t,$$

δ_t 表示第 t 天基于各玩家当前收益状态构造的阶段稳定性指标。

六、模型的评价

6.1 优点

1. 采用 Dijkstra 算法对地图进行预处理，简化路径，避免计算量过大；
2. 通过负重重量进行回溯路径、采购数量等，记录方案详细内容，从而找到最优解；
3. 第二问中引入马尔可夫模型参与决策，引入概率这一因素建立了基于期望的最大化收益模型，将不确定的问题通过随机确定的方式化解；
4. 第三问采用蒙特卡洛算法随机模拟天气，从而给出一般天气状况下的最佳方案；

5. 第三问先进行数值分析，确定合作而非竞争的博弈关系，将多人问题最优解降维为单人最优路径规划，并引入联盟稳定性指标进行验证。

6.2 缺点

1. 运用蒙特卡洛进行天气预测时，必须给定各天气概率，预测结果准确性有待提升；
2. 问题三采用整体收益最大化为目标，个体收益不一定最大，且未考虑多人错峰挖矿的情况；
3. 问题三仅限于小基数玩家的讨论，未考虑大基数玩家的情况，模型普适性有待提高。

参考文献

- [1] Hamiden Abd El-Wahed Khalifa and Pavan Kumar. Multi-objective optimisation for solving cooperative continuous static games using karush-kuhn-tucker conditions. *International Journal of Operational Research*, 46(1):133–147, 2023. Article ID: 128544.
- [2] Nadav Levy, Ido Klein, and Eran Ben-Elia. Emergence of cooperation and a fair system optimum in road networks: A game-theoretic and agent-based modelling approach. *Research in Transportation Economics*, 68:46–55, 2018. SI: Frontiers in Transportation.
- [3] On Park, {Hyo Sang} Shin, and Antonious Thourdos. Evolutionary game theory based multi-objective optimization for control allocation of over-actuated system. *IFAC Proceedings Volumes (IFAC-PapersOnline)*, 52(12):310–315, October 2019. Publisher Copyright: Copyright © 2019. The Authors. Published by Elsevier Ltd. All rights reserved.; 21st IFAC Symposium on Automatic Control in Aerospace, ACA 2019 ; Conference date: 27-08-2019 Through 30-08-2019.
- [4] Ireneusz Szczęśniak and Bożena Woźna-Szczęśniak. Generic dijkstra: correctness and tractability. In *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*, pages 1–7, 2023.
- [5] 董正华, 姜英姿, and 燕善俊. 基于深度 dp 搜索的穿越沙漠问题的研究. *现代信息技术*, 6(2):111–113, 2022.

附录：程序代码

第一关代码

```

#include<bits/stdc++.h>
using namespace std;
int dp[31][31][410][610]; // i, j, k, l, 时间, 地点, 水量, 食物。
int daystate[35] = {0, 2, 2, 1, 3, 1, 2, 3, 1, 2, 2,
    3, 2, 1, 2, 2, 2, 3, 3, 2, 2,
    1, 1, 2, 1, 3, 2, 1, 1, 2, 2}; // 1, 2, 3 晴朗, 高温, 沙暴;
int wa[5] = {0, 5, 8, 10};
int fo[5] = {0, 7, 6, 10};
vector<int> v[35];
void sta(){

    v[27].push_back(26);
    v[27].push_back(21);

    v[26].push_back(27);
    v[26].push_back(25);

    v[25].push_back(24);
    v[25].push_back(1);
    v[25].push_back(26);

    v[24].push_back(25);
    v[24].push_back(23);

    v[23].push_back(24);
    v[23].push_back(21);

    v[21].push_back(23);
    v[21].push_back(9);
    v[21].push_back(27);

    v[9].push_back(21);
    v[9].push_back(10);
    v[9].push_back(15);

    v[10].push_back(9);
    v[10].push_back(11);

    v[11].push_back(10);
    v[11].push_back(12);

    v[12].push_back(13);
    v[12].push_back(11);

    v[13].push_back(12);
    v[13].push_back(15);

```

```

v[15].push_back(9);
v[15].push_back(13);

v[1].push_back(25);
}

void dfs(int t, int cur, int w, int f){
    if(t == 0){
        printf("决策: 移动, 当天的天气%d 第几天: %d %d %d %d %d\n", daystate[t], t, cur, w, f,
            ↪ dp[t][cur][w][f]);
        return;
    }
    if(daystate[t] != 3){
        if(cur != 15){ //移动
            for(auto p : v[cur])
                if(dp[t - 1][p][w + 2 * wa[daystate[t]]][f + 2 * fo[daystate[t]]] ==
                    ↪ dp[t][cur][w][f]){
                    dfs(t - 1, p, w + 2 * wa[daystate[t]], f + 2 * fo[daystate[t]]);
                    printf("决策: 移动, 当天的天气%d 第几天: %d %d %d %d %d\n", daystate[t], t,
                        ↪ cur, w, f, dp[t][cur][w][f]);
                    return;
                }
        }
        else {
            for(auto p : v[cur])
                for(int k = 2 * wa[daystate[t]]; k <= w + 2 * wa[daystate[t]]; k++) // 要走一
                    ↪ 步
                        for(int l = 2 * fo[daystate[t]]; l <= f + 2 * fo[daystate[t]]; l++)
                            if(dp[t - 1][p][k][l] == dp[t][cur][w][f] +
                                10 * (w + 2 * wa[daystate[t]] - k) + 20 * (f + 2 * fo[daystate[t]]
                                    ↪ - 1)){
                                dfs(t - 1, p, k, l);
                                printf("决策: 买东西, 当天的天气%d 第几天: %d %d %d %d %d\n",
                                    ↪ daystate[t], t, cur, w, f, dp[t][cur][w][f]);
                                return;
                            }
        }
    }
}

if(cur == 12)
    if(dp[t - 1][cur][w + 3 * wa[daystate[t]]][f + 3 * fo[daystate[t]]] ==
        ↪ dp[t][cur][w][f] - 1000){
        dfs(t - 1, cur, w + 3 * wa[daystate[t]], f + 3 * fo[daystate[t]]);
        printf("决策: 挖矿, 当天的天气%d 第几天: %d %d %d %d %d\n", daystate[t], t, cur, w,
            ↪ f, dp[t][cur][w][f]);
        return;
    }

if(dp[t - 1][cur][w + 1 * wa[daystate[t]]][f + 1 * fo[daystate[t]]] == dp[t][cur][w][f]){
    dfs(t - 1, cur, w + 1 * wa[daystate[t]], f + 1 * fo[daystate[t]]);
}

```

```

        printf("决策: 停留, 当天的天气%d 第几天: %d %d %d %d %d\n", daystate[t], t, cur, w, f,
        ↪ dp[t][cur][w][f]);
        return;
    }
}

int main(){
    sta();

    for(int i = 0; i <= 30; i++)
        for(int j = 0; j <= 30; j++)
            for(int k = 0; k <= 400; k++)
                for(int l = 0; l <= 600; l++)
                    dp[i][j][k][l] = -5e3;

    for(int k = 0; k <= 400; k++){
        for(int l = 0; l <= (1200 - 3 * k) / 2; l++){
            if(10000 - 5 * k - 10 * l >= 0)
                dp[0][1][k][l] = 10000 - 5 * k - 10 * l;
        }
    }

    for(int i = 1; i <= 30; i++){
        if(daystate[i] != 3) //非沙暴天
            for(int j = 1; j <= 27; j++)
                for(int k = wa[daystate[i]] * 2; k <= 400; k++)
                    for(int l = fo[daystate[i]] * 2; l <= (1200 - 3 * k) / 2; l++){
                        if(dp[i - 1][j][k][l] < 0) continue;
                        for(auto t : v[j])
                            dp[i][t][k - wa[daystate[i]] * 2][l - fo[daystate[i]] * 2] =
                                max(dp[i - 1][j][k][l], dp[i][t][k - wa[daystate[i]] * 2][l -
                                ↪ fo[daystate[i]] * 2]);
                            //移动
                    }

            for(int j = 1; j <= 27; j++)
                for(int k = wa[daystate[i]]; k <= 400; k++)
                    for(int l = fo[daystate[i]]; l <= (1200 - 3 * k) / 2; l++){
                        if(dp[i - 1][j][k][l] < 0) continue;
                        dp[i][j][k - wa[daystate[i]]][l - fo[daystate[i]]] =
                            max(dp[i - 1][j][k][l], dp[i][j][k - wa[daystate[i]]][l -
                            ↪ fo[daystate[i]]]);
                            //停留
                    }

            for(int k = wa[daystate[i]] * 3; k <= 400; k++)
                for(int l = fo[daystate[i]] * 3; l <= (1200 - 3 * k) / 2; l++){

```

```

        if(dp[i - 1][12][k][1] < 0) continue;
        dp[i][12][k - wa[daystate[i]] * 3][1 - fo[daystate[i]] * 3] =
            max(dp[i - 1][12][k][1] + 1000, dp[i][12][k - wa[daystate[i]] * 3][1 -
            ↪ fo[daystate[i]] * 3]);
        //挖矿
    }

    for(int k = 0; k <= 400; k++)
        for(int l = 0; l <= (1200 - 3 * k) / 2; l++){
            if(k > 0)
                dp[i][15][k][1] = max(dp[i][15][k][1], dp[i][15][k - 1][1] - 10); //价格双
                ↪ 倍
            if(l > 0)
                dp[i][15][k][1] = max(dp[i][15][k][1], dp[i][15][k][1 - 1] - 20);
        } //买东西
    }

    int res = 0;
    int t = 0;
    for(int i = 1; i <= 30; i++){
        if(res < dp[i][27][0][0]){
            res = dp[i][27][0][0];
            t = i;
        }
    }
    dfs(t, 27, 0, 0);
    cout << res << " " << t << endl;
}

```

第二关代码

```

#pragma GCC optimize("O2")
#pragma GCC optimize("Ofast")
#pragma GCC optimize("inline")
#pragma GCC optimize("omit-frame-pointer")
#pragma GCC optimize("unroll-loops")
#include<bits/stdc++.h>
using namespace std;
int Po = 27;
int dp[31][65][403][603]; // i, j, k, l, 时间, 地点, 水量, 食物。
int daystate[35] = {0, 2, 2, 1, 3, 1, 2, 3, 1, 2, 2,
    3, 2, 1, 2, 2, 2, 3, 3, 2, 2,
    1, 1, 2, 1, 3, 2, 1, 1, 2, 2}; // 1, 2, 3 晴朗, 高温, 沙暴;
int wa[5] = {0, 5, 8, 10};
int fo[5] = {0, 7, 6, 10};
vector<int> v[70];

void sta(){
    //第二关数据
    for(int i = 1; i <= 63; i++)

```

```

        if(i % 8){
            v[i].push_back(i + 1);
            v[i + 1].push_back(i);
        }

for(int i = 1; i <= 56; i++){
    v[i].push_back(i + 8);
    v[i + 8].push_back(i);
}

for(int i = 0; i <= 3; i++){
    for(int j = 2; j <= 8; j++){
        v[i * 16 + j].push_back(i * 16 + j + 7);
        v[i * 16 + j + 7].push_back(i * 16 + j);
    }
for(int i = 1; i <= 3; i++){
    for(int j = 1; j <= 7; j++){
        v[i * 16 - (8 - j)].push_back(i * 16 - (8 - j) + 9);
        v[i * 16 - (8 - j) + 9].push_back(i * 16 - (8 - j));
    }
}
}

void dfs(int t, int cur, int w, int f){
    if(t == 0){
        printf("决策: 移动, 当天的天气%d 第几天: %d %d %d %d %d\n", daystate[t], t, cur, w, f,
            ↪ dp[t][cur][w][f]);
        return;
    }
    if(daystate[t] != 3){
        if(cur != 39 && cur != 62){ //移动
            for(auto p : v[cur])
                if(dp[t - 1][p][w + 2 * wa[ daystate[t] ]][f + 2 * fo[ daystate[t] ]] ==
                    ↪ dp[t][cur][w][f]){
                    dfs(t - 1, p, w + 2 * wa[ daystate[t] ], f + 2 * fo[ daystate[t] ]);
                    printf("决策: 移动, 当天的天气%d 第几天: %d %d %d %d %d\n", daystate[t], t,
                        ↪ cur, w, f, dp[t][cur][w][f]);
                    return;
                }
        }
    }else{
        for(auto p : v[cur])
            for(int k = 2 * wa[daystate[t]]; k <= w + 2 * wa[daystate[t]]; k++) // 要走一
                ↪ 步
                    for(int l = 2 * fo[daystate[t]]; l <= f + 2 * fo[daystate[t]]; l++)
                        if(dp[t - 1][p][k][l] == dp[t][cur][w][f] +
                            10 * (w + 2 * wa[daystate[t]] - k) + 20 * (f + 2 * fo[daystate[t]]
                                ↪ - 1)){

```

```

        dfs(t - 1, p, k, l);
        printf("决策: 买东西, 当天的天气%d 第几天: %d %d %d %d %d\n",
            ↪ daystate[t], t, cur, w, f, dp[t][cur][w][f]);
        return;
    }
}

if(cur == 30 || cur == 55)
    if(dp[t - 1][cur][w + 3 * wa[daystate[t]]][f + 3 * fo[daystate[t]]] ==
        ↪ dp[t][cur][w][f] - 1000){
        dfs(t - 1, cur, w + 3 * wa[daystate[t]], f + 3 * fo[daystate[t]]);
        printf("决策: 挖矿, 当天的天气%d 第几天: %d %d %d %d %d\n", daystate[t], t, cur, w,
            ↪ f, dp[t][cur][w][f]);
        return;
    }

if(dp[t - 1][cur][w + 1 * wa[daystate[t]]][f + 1 * fo[daystate[t]]] == dp[t][cur][w][f]){
    dfs(t - 1, cur, w + 1 * wa[daystate[t]], f + 1 * fo[daystate[t]]);
    printf("决策: 停留, 当天的天气%d 第几天: %d %d %d %d %d\n", daystate[t], t, cur, w, f,
        ↪ dp[t][cur][w][f]);
    return;
}

}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    sta();

    for(int i = 0; i <= 30; i++){
        for(int j = 0; j <= 64; j++){
            for(int k = 0; k <= 400; k++){
                for(int l = 0; l <= 600; l++){
                    dp[i][j][k][l] = -5e3;
                }
            }
        }
    }

    for(int k = 0; k <= 400; k++){
        for(int l = 0; l <= (1200 - 3 * k) / 2; l++){
            if(10000 - 5 * k - 10 * l >= 0)
                dp[0][1][k][l] = 10000 - 5 * k - 10 * l;
        }
    }

    for(int i = 1; i <= 30; i++){
        if(daystate[i] != 3) //非沙暴天
            for(int j = 1; j <= 64; j++){
                for(int k = wa[daystate[i]] * 2; k <= 400; k++)

```



```

        for(int l = fo[daystate[i]] * 2; l <= (1200 - 3 * k) / 2; l++){
            if(dp[i - 1][j][k][l] < 0) continue;
            for(auto t : v[j])
                dp[i][t][k - wa[daystate[i]] * 2][l - fo[daystate[i]] * 2] =
                    max(dp[i - 1][j][k][l], dp[i][t][k - wa[daystate[i]] * 2][l -
                        ↪ fo[daystate[i]] * 2]);
                //移动
        }

    for(int j = 1; j <= 64; j++)
        for(int k = wa[daystate[i]]; k <= 400; k++)
            for(int l = fo[daystate[i]]; l <= (1200 - 3 * k) / 2; l++){
                if(dp[i - 1][j][k][l] < 0) continue;
                dp[i][j][k - wa[daystate[i]]][l - fo[daystate[i]]] =
                    max(dp[i - 1][j][k][l], dp[i][j][k - wa[daystate[i]]][l -
                        ↪ fo[daystate[i]]]);
                //停留
            }

    for(int k = wa[daystate[i]] * 3; k <= 400; k++)
        for(int l = fo[daystate[i]] * 3; l <= (1200 - 3 * k) / 2; l++){
            if(dp[i - 1][30][k][l] < 0) continue;
            dp[i][30][k - wa[daystate[i]] * 3][l - fo[daystate[i]] * 3] =
                max(dp[i - 1][30][k][l] + 1000, dp[i][12][k - wa[daystate[i]] * 3][l -
                    ↪ fo[daystate[i]] * 3]);
                //挖矿
        }

    for(int k = wa[daystate[i]] * 3; k <= 400; k++)
        for(int l = fo[daystate[i]] * 3; l <= (1200 - 3 * k) / 2; l++){
            if(dp[i - 1][55][k][l] < 0) continue;
            dp[i][55][k - wa[daystate[i]] * 3][l - fo[daystate[i]] * 3] =
                max(dp[i - 1][55][k][l] + 1000, dp[i][55][k - wa[daystate[i]] * 3][l -
                    ↪ fo[daystate[i]] * 3]);
                //挖矿
        }

    for(int k = 0; k <= 400; k++)
        for(int l = 0; l <= (1200 - 3 * k) / 2; l++){
            if(k > 0)
                dp[i][39][k][l] = max(dp[i][39][k][l], dp[i][39][k - 1][l] - 10); //价格双
                ↪ 倍
            if(l > 0)
                dp[i][39][k][l] = max(dp[i][39][k][l], dp[i][39][k][l - 1] - 20);
        } //买东西
    for(int k = 0; k <= 400; k++)
        for(int l = 0; l <= (1200 - 3 * k) / 2; l++){
            if(k > 0)

```

```

        dp[i][62][k][l] = max(dp[i][62][k][l], dp[i][62][k-1][l] - 10); //价格双
        ↪ 倍
        if(l > 0)
            dp[i][62][k][l] = max(dp[i][62][k][l], dp[i][62][k][l-1] - 20);
    } //买东西
}

int res = 0;
int t = 0;
for(int i = 1; i <= 30; i++){
    if(res < dp[i][64][0][0]){
        res = dp[i][64][0][0];
        t = i;
    }
}

dfs(t, 64, 0, 0);

cout << res << " " << t << endl;
}

```

第三关代码

```

#pragma GCC optimize("O2")
#pragma GCC optimize("Ofast")
#pragma GCC optimize("inline")
#pragma GCC optimize("omit-frame-pointer")
#pragma GCC optimize("unroll-loops")
#include<bits/stdc++.h>
using namespace std;
int dp[20][20][403][603]; // i, j, k, l, 时间, 地点, 水量, 食物。
int daystate[35] = {0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2}; // 1, 2, 3 晴朗, 高温, 沙暴;
int wa[5] = {0, 3, 9, 10};
int fo[5] = {0, 4, 9, 10};
vector<int> v[30];
int comes = 1, wt, fd;
void sta(){

    // 第三关数据
    v[1].push_back(2), v[1].push_back(4), v[1].push_back(5);
    v[2].push_back(1), v[2].push_back(3), v[2].push_back(4);
    v[3].push_back(2), v[3].push_back(4), v[3].push_back(8), v[3].push_back(9);
    v[4].push_back(1), v[4].push_back(2), v[4].push_back(3),
    v[4].push_back(5), v[4].push_back(6), v[4].push_back(7);
    v[5].push_back(1), v[5].push_back(4), v[5].push_back(6);
    v[6].push_back(4), v[6].push_back(5), v[6].push_back(7),
    v[6].push_back(12), v[6].push_back(13);
    v[7].push_back(4), v[7].push_back(11), v[7].push_back(6), v[7].push_back(12);
    v[8].push_back(3), v[8].push_back(9);
}

```

```

v[9].push_back(3), v[9].push_back(8), v[9].push_back(10), v[9].push_back(11);
v[10].push_back(9), v[10].push_back(11), v[10].push_back(13);
v[11].push_back(7), v[11].push_back(9), v[11].push_back(10),
v[11].push_back(12), v[11].push_back(13);
v[12].push_back(6), v[12].push_back(7), v[12].push_back(11), v[12].push_back(13);
v[13].push_back(6), v[13].push_back(10), v[13].push_back(11), v[13].push_back(12);

}

struct S{
    int t, day, wat, foo, re;
}st[40];
int cnt = 0;

void dfs(int t, int cur, int w, int f, int now){
    if(now == t){
        wt = w;
        fd = f;
        comes = cur;
        st[cnt++] = {t, cur, w, f, dp[t][cur][w][f]};
        return;
    }

    if(t == 0){
        st[cnt++] = {t, cur, w, f, dp[t][cur][w][f]};
        // printf("%d %d %d %d %d\n", t, cur, w, f, dp[t][cur][w][f]);
        return;
    }

    if(daystate[t] != 3){
        if(cur != 20 && cur != 20){ //移动
            for(auto p : v[cur])
                if(dp[t - 1][p][w + 2 * wa[daystate[t]]][f + 2 * fo[daystate[t]]] ==
                    dp[t][cur][w][f]){
                    dfs(t - 1, p, w + 2 * wa[daystate[t]], f + 2 * fo[daystate[t]], now);
                    // printf("%d %d %d %d %d\n", t, cur, w, f, dp[t][cur][w][f]);
                    return;
                }
        }
        }else{
            for(auto p : v[cur])
                for(int k = 2 * wa[daystate[t]]; k <= w + 2 * wa[daystate[t]]; k++) // 要走一
                    步
                        for(int l = 2 * fo[daystate[t]]; l <= f + 2 * fo[daystate[t]]; l++)
                            if(dp[t - 1][p][k][l] == dp[t][cur][w][f] +
                                10 * (w + 2 * wa[daystate[t]] - k) + 20 * (f + 2 * fo[daystate[t]]
                                    - 1)){
                                    dfs(t - 1, p, k, l, now);
                                    // printf("%d %d %d %d %d\n", t, cur, w, f, dp[t][cur][w][f]);
                                }
        }
    }
}

```

```

        return;
    }
}

}

if(cur == 9)
    if(dp[t - 1][cur][w + 3 * wa[daystate[t]]][f + 3 * fo[daystate[t]]] ==
        ⇨ dp[t][cur][w][f] - 200){
        dfs(t - 1, cur, w + 3 * wa[daystate[t]] , f + 3 * fo[daystate[t]], now);
        // printf("%d %d %d %d %d\n", t, cur, w, f, dp[t][cur][w][f]);
        return;
    }

if(dp[t - 1][cur][w + 1 * wa[daystate[t]]][f + 1 * fo[daystate[t]]] == dp[t][cur][w][f]){
    dfs(t - 1, cur, w + 1 * wa[daystate[t]] , f + 1 * fo[daystate[t]], now);
    // printf("%d %d %d %d %d\n", t, cur, w, f, dp[t][cur][w][f]);
    return;
}

}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    auto start = std::chrono::high_resolution_clock::now();
    sta();

    double trans_prob[3][3];
    trans_prob[1][1] = 0.5;    // 晴 → 晴: 2 次
    trans_prob[1][2] = 0.5;    // 晴 → 高: 6 次
    trans_prob[1][3] = 0.0;     // 晴 → 沙: 0

    // 高温 (2) 的转移 (只考虑晴朗和高温)
    trans_prob[2][1] = 0.2;    // 高 → 晴: 5 次
    trans_prob[2][2] = 0.8;    // 高 → 高: 10 次
    trans_prob[2][3] = 0.0;     // 高 → 沙: 0

    // 沙暴 (3) 的转移 (实际不会用到)
    trans_prob[3][1] = 0.0;
    trans_prob[3][2] = 0.0;
    trans_prob[3][3] = 0.0;
    srand(time(0));

    for(int day = 1; day <= 10; day++){

        double ran = rand() % 50000 / (double)50000;
        if(daystate[day - 1] == 0){
            if(trans_prob[1][1] >= ran) {

```

```

        daystate[day] = 1; // 晴天
    }else
        daystate[day] = 2; // 高温
    }else {
        if(trans_prob[daystate[day - 1]][1] >= ran) // 根据上一天的天气来看
            daystate[day] = 1; // 晴天
        else
            daystate[day] = 2; // 高温
    }
    // daystate[day] = 2;
    for(int i = 0; i <= 10; i++)
        for(int j = 0; j <= 13; j++)
            for(int k = 0; k <= 400; k++)
                for(int l = 0; l <= 600; l++)
                    if(i != day - 1 || comes != j || day == 1 || wt != k || fd != l)
                        dp[i][j][k][l] = -5e3;
    if(day == 1){
        for(int k = 0; k <= 400; k++){
            for(int l = 0; l <= (1200 - 3 * k) / 2; l++){
                if(10000 - 5 * k - 10 * l >= 0)
                    dp[0][1][k][l] = 10000 - 5 * k - 10 * l;
            }
        }
    }
}

for(int i = 1; i <= 10; i++){
    if(daystate[i] != 3) //非沙暴天
        for(int j = 1; j <= 13; j++)
            for(int k = wa[daystate[i]] * 2; k <= 400; k++)
                for(int l = fo[daystate[i]] * 2; l <= (1200 - 3 * k) / 2; l++){
                    if(dp[i - 1][j][k][l] < 0) continue;
                    for(auto t : v[j])
                        dp[i][t][k - wa[daystate[i]] * 2][l - fo[daystate[i]] * 2] =
                            max(dp[i - 1][j][k][l], dp[i][t][k - wa[daystate[i]] * 2][l -
                                ↪ fo[daystate[i]] * 2]);
                    //移动
                }

    for(int j = 1; j <= 13; j++)
        for(int k = wa[daystate[i]]; k <= 400; k++)
            for(int l = fo[daystate[i]]; l <= (1200 - 3 * k) / 2; l++){
                if(dp[i - 1][j][k][l] < 0) continue;
                dp[i][j][k - wa[daystate[i]]][l - fo[daystate[i]]] =
                    max(dp[i - 1][j][k][l], dp[i][j][k - wa[daystate[i]]][l -
                        ↪ fo[daystate[i]]]);
                //停留
            }
}

```

```

    }

    for(int k = wa[daystate[i]] * 3; k <= 400; k++)
        for(int l = fo[daystate[i]] * 3; l <= (1200 - 3 * k) / 2; l++){
            if(dp[i - 1][9][k][l] < 0) continue;
            dp[i][9][k - wa[daystate[i]] * 3][l - fo[daystate[i]] * 3] =
                max(dp[i - 1][9][k][l] + 200, dp[i][9][k - wa[daystate[i]] * 3][l -
                    ↪ fo[daystate[i]] * 3]);
                //挖矿
        }

    // for(int k = 0; k <= 400; k++)
    //     for(int l = 0; l <= (1200 - 3 * k) / 2; l++){
    //         if(k > 0)
    //             dp[i][39][k][l] = max(dp[i][39][k][l], dp[i][39][k - 1][l] - 10); //价
    //         ↪ 格双倍
    //         if(l > 0)
    //             dp[i][39][k][l] = max(dp[i][39][k][l], dp[i][39][k][l - 1] - 20);
    //     } //买东西
}

double res = 0;
int t = 0;
int k1 = 0, l1 = 0;
for(int i = 1; i <= 10; i++)
    for(int k = 0; k <= 400; k++)
        for(int l = 0; l <= 600; l++)
            if(res < dp[i][13][k][l]){
                res = dp[i][13][k][l];
                k1 = k;
                l1 = l;
                t = i;
            }
dfs(t, 13, k1, l1, day);
if(t == day) break;
}

for(int i = 0; i < cnt; i++){
    if(i == cnt - 1){
        st[i].re += st[i].wat * 2.5 + st[i].foo * 5.0;
        st[i].wat = st[i].foo = 0;
    }

    cout << st[i].t << " " << st[i].day << " " << st[i].wat << " " << st[i].foo << " " <<
        ↪ st[i].re << endl;
}

for(int i = 0; i <= 10; i++)
    cout << daystate[i] << " ";

```



```

int cnt = 0;

void dfs(int t, int cur, int w, int f, int now){
    if(now == t){
        wt = w;
        fd = f;
        comes = cur;
        st[cnt++] = {t, cur, w, f, dp[t][cur][w][f]};
        return;
    }

    if(t == 0){
        st[cnt++] = {t, cur, w, f, dp[t][cur][w][f]};
        // printf("%d %d %d %d %d\n", t, cur, w, f, dp[t][cur][w][f]);
        return;
    }
    if(daystate[t] != 3){
        if(cur != 14){ //移动
            for(auto p : v[cur])
                if(dp[t - 1][p][w + 2 * wa[daystate[t]]][f + 2 * fo[daystate[t]]] ==
                    dp[t][cur][w][f]){
                    dfs(t - 1, p, w + 2 * wa[daystate[t]], f + 2 * fo[daystate[t]], now);
                    // printf("%d %d %d %d %d\n", t, cur, w, f, dp[t][cur][w][f]);
                    return;
                }
        }
        }else{
            for(auto p : v[cur])
                for(int k = 2 * wa[daystate[t]]; k <= w + 2 * wa[daystate[t]]; k++) // 要走一
                    ↪ 步
                        for(int l = 2 * fo[daystate[t]]; l <= f + 2 * fo[daystate[t]]; l++)
                            if(dp[t - 1][p][k][l] == dp[t][cur][w][f] +
                                10 * (w + 2 * wa[daystate[t]] - k) + 20 * (f + 2 * fo[daystate[t]]
                                    ↪ - 1)){
                                    dfs(t - 1, p, k, l, now);
                                    // printf("%d %d %d %d %d\n", t, cur, w, f, dp[t][cur][w][f]);
                                    return;
                                }
                            }
        }

    }
    if(cur == 18)
        if(dp[t - 1][cur][w + 3 * wa[daystate[t]]][f + 3 * fo[daystate[t]]] ==
            dp[t][cur][w][f] - 1000){
            dfs(t - 1, cur, w + 3 * wa[daystate[t]], f + 3 * fo[daystate[t]], now);
            // printf("%d %d %d %d %d\n", t, cur, w, f, dp[t][cur][w][f]);
            return;
        }
}

```



```

    if(dp[t - 1][cur][w + 1 * wa[daystate[t]]][f + 1 * fo[daystate[t]]] == dp[t][cur][w][f]){
        dfs(t - 1, cur, w + 1 * wa[daystate[t]] , f + 1 * fo[daystate[t]], now);
        // printf("%d %d %d %d %d\n", t, cur, w, f, dp[t][cur][w][f]);
        return;
    }
}

}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    auto start = std::chrono::high_resolution_clock::now();
    sta();

    double trans_prob[3][3];
    trans_prob[1][1] = 0.5;    // 晴 → 晴: 2 次
    trans_prob[1][2] = 0.4;    // 晴 → 高: 6 次
    trans_prob[1][3] = 0.1;    // 晴 → 沙: 0

    // 高温 (2) 的转移 (只考虑晴朗和高温)
    trans_prob[2][1] = 0.3;    // 高 → 晴: 5 次
    trans_prob[2][2] = 0.6;    // 高 → 高: 10 次
    trans_prob[2][3] = 0.1;    // 高 → 沙: 0

    // 沙暴 (3) 的转移 (实际不会用到)
    trans_prob[3][1] = 0.6;
    trans_prob[3][2] = 0.3;
    trans_prob[3][3] = 0.1;
    srand(time(0));

    for(int day = 1; day <= 30; day++){

        double ran = rand() % 50000 / (double)50000;
        if(daystate[day - 1] == 0){
            if(trans_prob[1][1] >= ran) {

                daystate[day] = 1; // 晴天
            }else if(trans_prob[1][1] + trans_prob[1][2] >= ran)
                daystate[day] = 2; // 高温
            else
                daystate[day] = 3; // 沙暴
        }else {
            if(trans_prob[daystate[day - 1]][1] >= ran) // 根据上一天的天气来看
                daystate[day] = 1; // 晴天
            else if(trans_prob[daystate[day - 1]][1] + trans_prob[daystate[day - 1]][2] >= ran)
                daystate[day] = 2; // 高温
            else

```

```

        daystate[day] = 3; //沙暴
    }
    // daystate[day] = 2;
    for(int i = 0; i <= 30; i++)
        for(int j = 0; j <= 25; j++)
            for(int k = 0; k <= 400; k++)
                for(int l = 0; l <= 600; l++)
                    if(i != day - 1 || comes != j || day == 1 || wt != k || fd != l)
                        dp[i][j][k][l] = -5e3;
    if(day == 1){
        for(int k = 0; k <= 400; k++){
            for(int l = 0; l <= (1200 - 3 * k) / 2; l++){
                if(10000 - 5 * k - 10 * l >= 0)
                    dp[0][1][k][l] = 10000 - 5 * k - 10 * l;
            }
        }
    }

    for(int i = 1; i <= 30; i++){
        if(daystate[i] != 3) //非沙暴天
            for(int j = 1; j <= 25; j++)
                for(int k = wa[daystate[i]] * 2; k <= 400; k++)
                    for(int l = fo[daystate[i]] * 2; l <= (1200 - 3 * k) / 2; l++){
                        if(dp[i - 1][j][k][l] < 0) continue;
                        for(auto t : v[j])
                            dp[i][t][k - wa[daystate[i]] * 2][l - fo[daystate[i]] * 2] =
                                max(dp[i - 1][j][k][l], dp[i][t][k - wa[daystate[i]] * 2][l -
                                    ↪ fo[daystate[i]] * 2]);
                            //移动
                    }

            for(int j = 1; j <= 25; j++)
                for(int k = wa[daystate[i]]; k <= 400; k++)
                    for(int l = fo[daystate[i]]; l <= (1200 - 3 * k) / 2; l++){
                        if(dp[i - 1][j][k][l] < 0) continue;
                        dp[i][j][k - wa[daystate[i]]][l - fo[daystate[i]]] =
                            max(dp[i - 1][j][k][l], dp[i][j][k - wa[daystate[i]]][l -
                                ↪ fo[daystate[i]]]);
                            //停留
                    }

            for(int k = wa[daystate[i]] * 3; k <= 400; k++)
                for(int l = fo[daystate[i]] * 3; l <= (1200 - 3 * k) / 2; l++){
                    if(dp[i - 1][18][k][l] < 0) continue;
                    dp[i][18][k - wa[daystate[i]] * 3][l - fo[daystate[i]] * 3] =

```

```

        max(dp[i - 1][18][k][1] + 1000, dp[i][18][k - wa[daystate[i]] * 3][1 -
        ↪ fo[daystate[i]] * 3]);
        //挖矿
    }

    for(int k = 0; k <= 400; k++)
        for(int l = 0; l <= (1200 - 3 * k) / 2; l++){
            if(k > 0)
                dp[i][14][k][1] = max(dp[i][14][k][1], dp[i][14][k - 1][1] - 10); //价格双
                ↪ 倍
            if(l > 0)
                dp[i][14][k][1] = max(dp[i][14][k][1], dp[i][14][k][l - 1] - 20);
        } //买东西
    }

    double res = 0;
    int t = 0;
    int k1 = 0, l1 = 0;
    for(int i = 1; i <= 25; i++)
        for(int k = 0; k <= 400; k++)
            for(int l = 0; l <= 600; l++)
                if(res < dp[i][25][k][1]){
                    res = dp[i][25][k][1];
                    k1 = k;
                    l1 = l;
                    t = i;
                }

    dfs(t, 25, k1, l1, day);
    if(t == day) break;
}

for(int i = 0; i < cnt; i++){
    if(i == cnt){
        st[i].re += st[i].wat * 2.5 + st[i].foo * 5.0;
        st[i].wat = st[i].foo = 0;
    }

    cout << st[i].t << " " << st[i].day << " " << st[i].wat << " " << st[i].foo << " " <<
    ↪ st[i].re << endl;
}

for(int i = 0; i <= 30; i++)
    cout << daystate[i] << " ";

auto end = std::chrono::high_resolution_clock::now();

std::chrono::duration<double> duration = end - start;

```

```
std::cout << "代码段运行时间: " << duration.count() << " 秒" << std::endl;
}
```

第六关代码

```
function result = desert_survival_no_supply()
clc; clearvars -except result;

%% =====
% 参数设置 (可按需要修改)
% =====
P.deadline = 30;      % 截止日期
P.moveDays = 8;      % 起点到终点所需 " 可移动日 " 数
P.maxLoad = 1200;    % 负重上限 kg
P.initMoney = 10000; % 初始资金 元

% 资源参数
P.waterMass = 3;      % 每箱水质量 kg
P.foodMass = 2;       % 每箱食物质量 kg
P.waterPrice = 5;     % 起点基准价格 元/箱
P.foodPrice = 10;     % 起点基准价格 元/箱

%% ===== 挖矿问题新增参数 =====
P.villageWaterPrice = 2 * P.waterPrice; % 村庄水价
P.villageFoodPrice = 2 * P.foodPrice;   % 村庄食物价

P.distStartMine = 5; % 起点-> 矿山
P.distMineVillage = 2; % 矿山-> 村庄 (单程)
P.distMineEnd = 3; % 矿山-> 终点
P.distVillageEnd = 3; % 村庄-> 终点

P.mineIncome = 1000; % 挖矿 1 天收益

% 补给次数序列: 0 表示不去村庄; 0.5 表示最后只去一次村庄后直接去终点;
% 1 表示矿山-> 村庄-> 矿山 完整一来回;
% 1.5 表示一来回后, 最后再去一次村庄并从村庄去终点
P.supplyCountList = 0:0.5:5; % 这个范围你可以自己改

% 基础消耗 (箱)
% [水, 食物]
P.baseClear = [3, 4];
P.baseHot = [9, 9];
P.baseSand = [10, 10];

% 行走倍数、停留倍数 (题目规则)
P.walkMult = 2; % 行走日 = 2 * 基础消耗
P.stayMult = 1; % 停留日 = 1 * 基础消耗
```

```

% 天气生成概率（这是“未知天气”下必须额外假设的部分，可自行改）
% 概率之和必须为 1

P.pSand      = 0.15;
P.pClear     = 0.40;
P.pHot       = 1-P.pClear-P.pSand;

% 额外限制（按你截图中的思路）
P.maxSandDays = 10;      % c <= 10
P.maxTry      = 2e5;     % 为了采样足够有效路径，最多尝试次数
P.Nsim        = 5000;    % 蒙特卡洛样本数

%% =====
% 计算“补给次数-最大盈利”关系
% =====
fprintf('正在计算“补给次数-最大盈利”关系...\n');

profitList = nan(size(P.supplyCountList));
bestMineDaysList = nan(size(P.supplyCountList));
bestInitWaterList = nan(size(P.supplyCountList));
bestInitFoodList = nan(size(P.supplyCountList));

for k = 1:length(P.supplyCountList)
    supplyCount = P.supplyCountList(k);
    fprintf('正在计算补给次数 = %.1f ...\n', supplyCount);

    [bestProfit, bestMineDays, bestW0, bestF0] = ...
        optimizeMiningPlanForSupplyCount(P, supplyCount);

    profitList(k) = bestProfit;
    bestMineDaysList(k) = bestMineDays;
    bestInitWaterList(k) = bestW0;
    bestInitFoodList(k) = bestF0;
end

%% 绘图
figure;
plot(P.supplyCountList, profitList, '-o', 'LineWidth', 1.5, 'MarkerSize', 6);
xlabel('去村庄补给次数');
ylabel('最大盈利');
title('去村庄补给次数与最大盈利的关系');
grid on;

%% 如需同时看看最优挖矿天数，可加第二张图
figure;
plot(P.supplyCountList, bestMineDaysList, '-s', 'LineWidth', 1.5, 'MarkerSize', 6);
xlabel('去村庄补给次数');
ylabel('最优挖矿天数');

```

```

title('去村庄补给次数与最优挖矿天数的关系');
grid on;

%% 输出结果
result.supplyCountList = P.supplyCountList;
result.profitList = profitList;
result.bestMineDaysList = bestMineDaysList;
result.bestInitWaterList = bestInitWaterList;
result.bestInitFoodList = bestInitFoodList;

%% 输出结果
result.sandProbList = sandProbList;
result.survivalRateList = survivalRateList;
result.bestWaterList = bestWaterList;
result.bestFoodList = bestFoodList;
end

%% =====
% 蒙特卡洛生成样本：每条样本表示一条"完整到达终点"的天气路径
% 输出：
%   needW(k), needF(k): 第 k 条样本到达终点所需的水/食物箱数
% =====
function [needW, needF, tripDays] = simulateDemands(P)

needW = zeros(P.Nsim,1);
needF = zeros(P.Nsim,1);
tripDays = zeros(P.Nsim,1);

cnt = 0;      % 已接受样本数
tries = 0;    % 总尝试次数

while cnt < P.Nsim && tries < P.maxTry
    tries = tries + 1;

    moved = 0;      % 已完成的可移动天数
    day = 0;        % 已经历总天数
    sandDays = 0;

    seq = [];       % 1= 晴朗, 2= 高温, 3= 沙暴
    w = 0; f = 0;

    valid = true;

    while day < P.deadline && moved < P.moveDays
        day = day + 1;
        wt = drawWeather(P);

```

```

if wt == 3
    % 沙暴: 必须停留
    sandDays = sandDays + 1;
    w = w + P.stayMult * P.baseSand(1);
    f = f + P.stayMult * P.baseSand(2);
elseif wt == 1
    % 晴朗: 前进
    moved = moved + 1;
    w = w + P.walkMult * P.baseClear(1);
    f = f + P.walkMult * P.baseClear(2);
else
    % 高温: 前进
    moved = moved + 1;
    w = w + P.walkMult * P.baseHot(1);
    f = f + P.walkMult * P.baseHot(2);
end

seq(end+1) = wt; %#ok<AGROW>

% 约束 1: 沙暴总数上限
if sandDays > P.maxSandDays
    valid = false;
    break;
end

% 约束 2: 到当前为止沙暴比例 < 1/3 (你图里的"较少出现沙暴")
if sandDays / day >= 1/3
    valid = false;
    break;
end

% 约束 3: 任意连续 j>=2 天, 沙暴比例 <= 1/2
if ~checkWindowConstraint(seq)
    valid = false;
    break;
end
end

% 必须在截止日前完成 moveDays 次有效移动
if valid && moved >= P.moveDays && day <= P.deadline
    cnt = cnt + 1;
    needW(cnt) = w;
    needF(cnt) = f;
    tripDays(cnt) = day;
end
end

% 截断未使用部分

```

```

needW    = needW(1:cnt);
needF    = needF(1:cnt);
tripDays = tripDays(1:cnt);

end

%% =====
% 搜索最优初始购买方案
% 生存条件: 初始水 >= 该样本所需水, 且初始食物 >= 该样本所需食物
% =====
function [bestW0, bestF0, bestRate] = searchBestPlan(needW, needF, P)

maxWaterByLoad = floor(P.maxLoad / P.waterMass);
maxFoodByLoad  = floor(P.maxLoad / P.foodMass);
maxWaterByMoney = floor(P.initMoney / P.waterPrice);
maxFoodByMoney  = floor(P.initMoney / P.foodPrice);

maxW = min(maxWaterByLoad, maxWaterByMoney);
maxF = min(maxFoodByLoad, maxFoodByMoney);

bestRate = -1;
bestW0 = 0;
bestF0 = 0;

for W0 = 0:maxW
    % 由资金、负重双重限制推 FO 上界
    maxF_load = floor((P.maxLoad - P.waterMass * W0) / P.foodMass);
    maxF_money = floor((P.initMoney - P.waterPrice * W0) / P.foodPrice);
    upperF = min([maxF, maxF_load, maxF_money]);

    if upperF < 0
        continue;
    end

    for F0 = 0:upperF
        alive = (W0 >= needW) & (F0 >= needF);
        rate = mean(alive);

        if rate > bestRate
            bestRate = rate;
            bestW0 = W0;
            bestF0 = F0;
        end
    end
end
end

end

```



```

%% =====
% 随机生成一天的天气
% 1= 晴朗, 2= 高温, 3= 沙暴
% =====

function wt = drawWeather(P)
r = rand;
if r < P.pClear
    wt = 1;
elseif r < P.pClear + P.pHot
    wt = 2;
else
    wt = 3;
end
end

%% =====
% 检查: 任意连续  $j \geq 2$  天, 沙暴比例  $\leq 1/2$ 
% 这里采用严格判定
% =====

function ok = checkWindowConstraint(seq)
n = length(seq);
ok = true;

for len = 2:n
    for s = 1:(n-len+1)
        window = seq(s:s+len-1);
        sandRatio = sum(window == 3) / len;
        if sandRatio > 1/2
            ok = false;
            return;
        end
    end
end

function [bestProfit, bestMineDays, bestW0, bestF0] = optimizeMiningPlanForSupplyCount(P,
↪ supplyCount)

bestProfit = -inf;
bestMineDays = NaN;
bestW0 = NaN;
bestF0 = NaN;

% 根据补给次数, 先确定固定路线所需移动天数
[travelDays, needFinalVillageToEnd] = getTravelDaysFromSupplyCount(P, supplyCount);

```

```

% 超过截止日期则直接不可行
if travelDays > P.deadline
    return;
end

% 能用于挖矿的最大天数
maxMineDays = P.deadline - travelDays;

% 初始可购买上限（受资金和负重限制）
maxWaterByLoad = floor(P.maxLoad / P.waterMass);
maxFoodByLoad = floor(P.maxLoad / P.foodMass);
maxWaterByMoney = floor(P.initMoney / P.waterPrice);
maxFoodByMoney = floor(P.initMoney / P.foodPrice);

maxW = min(maxWaterByLoad, maxWaterByMoney);
maxF = min(maxFoodByLoad, maxFoodByMoney);

% 枚举：初始水、初始食物、挖矿天数
for W0 = 0:maxW
    maxF_load = floor((P.maxLoad - P.waterMass * W0) / P.foodMass);
    maxF_money = floor((P.initMoney - P.waterPrice * W0) / P.foodPrice);
    upperF = min(maxF, min(maxF_load, maxF_money));

    if upperF < 0
        continue;
    end

    for F0 = 0:upperF
        initCost = W0 * P.waterPrice + F0 * P.foodPrice;

        % 初始购买就已经超预算
        if initCost > P.initMoney
            continue;
        end

        for mineDays = 0:maxMineDays
            ok = checkMiningPlanFeasible(W0, F0, mineDays, supplyCount, P);

            if ok
                profit = mineDays * P.mineIncome - initCost;

                if profit > bestProfit
                    bestProfit = profit;
                    bestMineDays = mineDays;
                    bestW0 = W0;
                    bestF0 = F0;
                end
            end
        end
    end
end

```

```

        end
    end
end
end

if isinf(bestProfit)
    bestProfit = NaN;
end

end

function [travelDays, needFinalVillageToEnd] = getTravelDaysFromSupplyCount(P, supplyCount)

% supplyCount:
% 0   = 起点-> 矿山-> 终点
% 0.5 = 起点-> 矿山-> 村庄-> 终点
% 1   = 起点-> 矿山-> 村庄-> 矿山-> 终点
% 1.5 = 起点-> 矿山-> 村庄-> 矿山-> 村庄-> 终点
% 2   = 起点-> 矿山-> 村庄-> 矿山-> 村庄-> 矿山-> 终点
% ...

nRound = floor(supplyCount);           % 完整往返次数 (矿山-> 村庄-> 矿山)
hasHalf = abs(supplyCount - nRound - 0.5) < 1e-9;

travelDays = P.distStartMine ...
    + nRound * (2 * P.distMineVillage);

if hasHalf
    % 最后一次只到村庄, 再从村庄到终点
    travelDays = travelDays + P.distMineVillage + P.distVillageEnd;
    needFinalVillageToEnd = true;
else
    % 直接从矿山到终点
    travelDays = travelDays + P.distMineEnd;
    needFinalVillageToEnd = false;
end

end

function ok = checkMiningPlanFeasible(W0, F0, mineDays, supplyCount, P)

ok = false;

% 用高温作为保守消耗标准
walkNeed = P.walkMult * P.baseHot;    % [水, 食物]
mineNeed = 3 * P.baseHot;              % 挖矿 1 天 [水, 食物]

nRound = floor(supplyCount);

```

```

hasHalf = abs(supplyCount - nRound - 0.5) < 1e-9;

% ----- 第 1 段: 起点 -> 矿山 -----
need1 = P.distStartMine * walkNeed;
W = W0 - need1(1);
F = F0 - need1(2);

if W < 0 || F < 0
    return;
end

% ----- 中间若干轮: 矿山挖矿 + 去村庄补给 + 返回矿山 -----
% 设总挖矿天数尽量平均分布到各次在矿山阶段
numMineStages = nRound + 1; % 完整往返 n 次时, 共有 n+1 个 " 在矿山可挖矿阶段 "
if hasHalf
    numMineStages = nRound + 1; % 最后半次前也仍有 n+1 个矿山阶段
end

mineDaysAlloc = floor(mineDays / numMineStages) * ones(1, numMineStages);
r = mod(mineDays, numMineStages);
mineDaysAlloc(1:r) = mineDaysAlloc(1:r) + 1;

for i = 1:nRound
    % 先在矿山挖矿
    W = W - mineDaysAlloc(i) * mineNeed(1);
    F = F - mineDaysAlloc(i) * mineNeed(2);
    if W < 0 || F < 0
        return;
    end

    % 从矿山走到村庄
    goNeed = P.distMineVillage * walkNeed;
    W = W - goNeed(1);
    F = F - goNeed(2);
    if W < 0 || F < 0
        return;
    end

    % 在村庄补给: 为了完成 " 回矿山 + 后续所有行程 ", 一次性补到刚好够且不超负重
    [futureW, futureF] = futureNeedAfterVillage(i, nRound, hasHalf, mineDaysAlloc, P,
        ↪ walkNeed, mineNeed);

    buyW = max(0, futureW - W);
    buyF = max(0, futureF - F);

    if buyW * P.waterMass + buyF * P.foodMass > P.maxLoad
        return;
    end
end

```

```

W = W + buyW;
F = F + buyF;

% 回矿山
backNeed = P.distMineVillage * walkNeed;
W = W - backNeed(1);
F = F - backNeed(2);
if W < 0 || F < 0
    return;
end
end

% ----- 最后一个矿山阶段 -----
W = W - mineDaysAlloc(end) * mineNeed(1);
F = F - mineDaysAlloc(end) * mineNeed(2);
if W < 0 || F < 0
    return;
end

% ----- 结束段：去终点 -----
if hasHalf
    % 矿山->村庄
    goNeed = P.distMineVillage * walkNeed;
    W = W - goNeed(1);
    F = F - goNeed(2);
    if W < 0 || F < 0
        return;
    end

    % 最后一次只到村庄，补给后直接去终点
    endNeed = P.distVillageEnd * walkNeed;
    buyW = max(0, endNeed(1) - W);
    buyF = max(0, endNeed(2) - F);

    if buyW * P.waterMass + buyF * P.foodMass > P.maxLoad
        return;
    end

    W = W + buyW;
    F = F + buyF;

    W = W - endNeed(1);
    F = F - endNeed(2);
    if W < 0 || F < 0
        return;
    end
end
else

```

```

    % 直接矿山-> 终点
    endNeed = P.distMineEnd * walkNeed;
    W = W - endNeed(1);
    F = F - endNeed(2);
    if W < 0 || F < 0
        return;
    end
end

ok = true;

end

function [futureW, futureF] = futureNeedAfterVillage(iRound, nRound, hasHalf, mineDaysAlloc,
↳ P, walkNeed, mineNeed)

% 站在 " 第 iRound 次到达村庄补给点 " 这个时刻,
% 计算从此刻开始到终点还至少需要多少水/食物

futureW = 0;
futureF = 0;

% 先回矿山
futureW = futureW + P.distMineVillage * walkNeed(1);
futureF = futureF + P.distMineVillage * walkNeed(2);

% 后续矿山阶段: 第 iRound+1, ..., 最后
for j = (iRound+1):length(mineDaysAlloc)
    futureW = futureW + mineDaysAlloc(j) * mineNeed(1);
    futureF = futureF + mineDaysAlloc(j) * mineNeed(2);

    if j < length(mineDaysAlloc)
        % 还要再去一次村庄并返回矿山
        futureW = futureW + 2 * P.distMineVillage * walkNeed(1);
        futureF = futureF + 2 * P.distMineVillage * walkNeed(2);
    end
end

% 最后一段去终点
if hasHalf
    futureW = futureW + P.distMineVillage * walkNeed(1) + P.distVillageEnd * walkNeed(1);
    futureF = futureF + P.distMineVillage * walkNeed(2) + P.distVillageEnd * walkNeed(2);
else
    futureW = futureW + P.distMineEnd * walkNeed(1);
    futureF = futureF + P.distMineEnd * walkNeed(2);
end
end

```

end